RPX

RPA/RPX v0.1

ISA 规范手册

Reformulation Proof Assembly
Reformulation Proof eXecutable

机器模型 · 指令集 · 文本汇编 · 二进制格式
Loader · 执行语义 · 异常与符合性

规范草案 D0
2026 年 10 月 9 日

本文定义待讨论的 v0.1 规范，不代表现有代码已经实现。
标注的修订提案与开放问题须在版本冻结前逐项决定。

阅读约定与文档状态

RPA 是 *Reformulation Proof Assembly*；RPX 是 *Reformulation Proof eXecutable*。`.rpa` 文件描述证明、映射和分支；`.rpx` 文件保存 proof VM 可直接读取的有序二进制指令流。本文讨论普通有理系数 MILP 的 reformulation correctness：给定源模型与目标模型，证明约定的可行性和目标关系。

本文的基线是本项目讨论中的 v0.1 草案。基线包括三种 fact、28 个 core opcode、两个执行方向、表达式池与结构化 case。为使一份手册能够独立阅读，D0 为原草案中尚未给出完整编码或状态规则的部分提供了明确的**修订提案**。这些提案是可供审阅的候选规范，不是已获批准的版本冻结决定。源代码中的现有功能不构成本规范的隐含实现。

规范性用语

“必须”“不得”“拒绝”描述符合此草案的实现所需的行为；“建议”描述推荐做法；“可以”描述允许的选择。数学语义、字节布局、操作数约束与成功条件是规范性内容。算法建议、内存示意、性能预期与工具输出样例是说明性内容。文档状态为草案并不允许 verifier 对已声明的规则采取宽松解释；它表示规则本身仍可在审阅后修改。

手册的边界

基础 v0.1 定义精确线性推导、整数性、取整、仿射映射、输出模型验证义务（obligation，定义见第一章）与结构化 split。本手册不要求 proof search、LP/MIP 求解器、JIT、任意跳转或一般图算法。指令 tape 是已经安排好验证顺序的证书。assembler 可以进行任意不可信的优化与搜索，最终产生的文件仍须逐条通过 kernel 检查。

批量编码与高层 theorem 被预留，但基础 v0.1 不接受其 opcode。MCM 的规范数学接口在第一章定义；独立的 MCM 文件序列化和完整 MPS 接入规范仍是开放问题 O-01、O-02。它们不会由 RPX loader 猜测或补全。

建议阅读顺序

数学与可信边界审阅从第一章和第二章开始。assembler 与 VM 实现者再阅读第三章、第四章及第五章。第六章定义文本层，第七章提供完整证明实例。附录列出待决提案、开放问题和版本冻结检查点，供后续逐项修订。

本草案的符合性范围

基础 verifier 实现全部 core 指令、已启用方向、完整格式检查和规定的 contract 检查。
assembler 产生符合格式的 RPX；任何 assembler 判断均不代替 verifier 的验证。
disassembler 对合法结构产生语义可重组的规范文本，不承担数学证明正确性的认证。
扩展 verifier 额外公布支持的扩展版本与可信 checker；不得把扩展支持混同于基础 v0.1。

没有部分 core 支持的“基础符合性”。实现阶段可以报告 `UNSUPPORTED`，但不能把未检查的程序报告为 `VERIFIED`。

目录

阅读约定与文档状态 **i**

规范性用语 . . . i

手册的边界 . . . i

建议阅读顺序 . . . i

本草案的符合性范围 . . . i

第一章 数学模型与合约 **1**

1.1 规范范围与模型边界 . . . 1

1.2 精确标量 . . . 1

1.3 规范化 MILP . . . 2

1.3.1 变量 . . . 2

1.3.2 约束行 . . . 2

1.3.3 目标函数 . . . 3

1.3.4 可行域与扩展最优值 . . . 3

1.4 规范化仿射表达式 . . . 3

1.5 事实类型 . . . 4

1.6 输入模型的静态事实 . . . 4

1.7 两个执行方向 . . . 4

1.8 目标关系与合约 profile . . . 5

1.8.1 方向性命题（资料性） . . . 5

1.8.2 可行性等价 . . . 5

1.8.3 最优值等价 . . . 5

1.8.4 合约标志的约束 . . . 6

1.9 摘要绑定与接受结果的含义 . . . 6

第二章 执行模型、作用域与验证义务 **7**

2.1 方向状态 . . . 7

2.2 事实寄存器与隐式编号 . . . 7

2.3 作用域和 FactRef 解析 . . . 8

2.3.1 分支假设 . . . 9

2.3.2 删除事实 . . . 9

2.4 输出映射状态 . . . 9

2.5 验证义务的生成 . . . 10

2.5.1 义务的表达式 . . . 10

2.5.2 EXACT 检查模式 . . . 11
2.5.3 完成义务 . . . 11
2.6 结构化分裂与状态事务 . . . 11
2.6.1 Block 终结规则 . . . 12
2.6.2 Yield 与 proposition merge . . . 12
2.6.3 义务汇合：交集 . . . 12
2.6.4 映射汇合与封存 . . . 12
2.7 不可达作用域 . . . 13
2.7.1 不可达代码的结构处理 . . . 13
2.8 RET 与方向接受条件 . . . 14
2.9 错误类别与资源结果 . . . 14
2.10 正确性不变式 . . . 15
2.11 执行成本与存储说明（资料性） . . . 15

第三章 核心指令集参考 16
3.1 公共编码与执行约定 . . . 16
3.1.1 指令头与字节序 . . . 16
3.1.2 操作数与寄存器定义 . . . 16
3.1.3 校验与提交 . . . 16
3.1.4 操作码总表 . . . 17
3.2 一般指令 . . . 17
3.2.1 NOP：无操作 . . . 17
3.2.2 RET：完成当前方向 . . . 18
3.2.3 DROP：释放事实区间 . . . 18
3.3 线性事实指令 . . . 18
3.3.1 LIN.LE：非负不等式组合 . . . 18
3.3.2 LIN.EQ：等式线性组合 . . . 19
3.3.3 EQ.PAIR：相反不等式构成等式 . . . 19
3.3.4 BOX.LE：声明变量界检查 . . . 19
3.3.5 WEAKEN.LE：常数放松 . . . 20
3.3.6 CLOSE.FALSE：关闭矛盾域 . . . 20
3.4 整数事实指令 . . . 20
3.4.1 INT.AFF：整数仿射组合 . . . 20
3.4.2 ROUND.HI：整数上界取整 . . . 21
3.4.3 ROUND.LO：整数下界取整 . . . 21
3.5 映射指令 . . . 22
3.5.1 映射指令的公共条件 . . . 22
3.5.2 MAP.COPY：复制输入变量 . . . 22

3.5.3 MAP.CONST：常数映射 . . . 22
3.5.4 MAP.AFF：仿射映射 . . . 22
3.6 模型义务检查指令 . . . 23
3.6.1 CHECK 的公共语义与 EXACT 模式 . . . 23
3.6.2 CHECK.ROW.HI：输出行上界 . . . 23
3.6.3 CHECK.ROW.LO：输出行下界 . . . 24
3.6.4 CHECK.ROW.EQ：输出等式行 . . . 24
3.6.5 CHECK.VAR.HI：输出变量上界 . . . 24
3.6.6 CHECK.VAR.LO：输出变量下界 . . . 24
3.6.7 CHECK.VAR.INT：输出变量整数性 . . . 25
3.6.8 CHECK.OBJ.EQ：目标相等 . . . 25
3.6.9 CHECK.OBJ.LE：目标不增 . . . 25
3.7 结构化分类与汇合指令 . . . 26
3.7.1 SPLIT 公共格式 . . . 26
3.7.2 SPLIT 公共汇合语义 . . . 26
3.7.3 SPLIT.BIN：显式二元变量分类 . . . 26
3.7.4 SPLIT.INT.THR：整数阈值分类 . . . 27
3.7.5 SPLIT.INT.RANGE：整数有限区间枚举 . . . 28
3.7.6 SPLIT.SIGN：实表达式符号分类 . . . 28
3.7.7 YIELD：分支事实返回 . . . 29
3.8 逐指令载荷尺寸速查 . . . 29

第四章 RPX 二进制文件格式 31
4.1 通用编码约定 . . . 31
4.2 Header . . . 31
4.2.1 Contract word . . . 32
4.3 Section directory . . . 32
4.4 ScalarRef 与 ScalarPool . . . 33
4.4.1 内联整数 . . . 33
4.4.2 Pool 记录 . . . 33
4.5 Expression pool . . . 34
4.5.1 ExprIndex . . . 34
4.5.2 ExprTerm . . . 34
4.6 FactRef . . . 35
4.7 Map section . . . 35
4.8 Instruction stream . . . 36
4.8.1 隐式结果编号 . . . 36
4.8.2 Split 内部字节范围 . . . 36

4.9 可选数据和版本边界 36

第五章 Loader、验证结果与可信边界 37

5.1 输入与输出 37
5.2 处理阶段 37
5.3 必需的结构检查 38
5.3.1 区间与整数 38
5.3.2 未执行区域 38
5.4 执行检查与提交原子性 38
5.5 诊断类别 38
5.6 资源预算 39
5.7 文件映射与稳定性 39
5.8 可信计算基与扩展 39

第六章 RPA 文本汇编与工具链 40

6.1 文件结构 40
6.1.1 编码与词法约定 40
6.1.2 顶层声明 40
6.2 精确数值字面量 42
6.2.1 允许的拼写 42
6.2.2 规范输出 42
6.3 操作数与符号 42
6.4 表达式声明与展开 43
6.4.1 仿射语法 43
6.4.2 指令中的 expression 操作数 43
6.5 静态 mapping 区域 44
6.6 基础指令的文本形式 44
6.6.1 General、linear 与 integer 指令 45
6.6.2 Mapping 与 obligation 指令 46
6.6.3 `exact` 简写 46
6.6.4 Structured cases 47
6.7 FactId、scope 与规范反汇编 48
6.8 核心 EBNF 48
6.9 工具链接口草案 51
6.10 扩展边界与保留语法 52

第七章 完整证书示例与验收案例 53

7.1 示例 A：删除冗余上界行 53
7.1.1 模型与 contract 53

7.1.2 完整 RPA 54
7.1.3 检查要点 54
7.2 示例 B：利用等式消去一个变量 55
7.2.1 模型与 contract 55
7.2.2 完整 RPA 56
7.2.3 检查要点 56
7.3 示例 C：分段 absolute-value mapping 57
7.3.1 模型与 contract 57
7.3.2 完整 RPA 58
7.3.3 检查要点与 mapping 封存 59
7.4 整数性与取整的小型推导片段 59
7.5 LIN.LE 的二进制 golden vector 59
7.6 失败证书与一致性验收 60

第八章 执行成本、实现建议与测量 62
8.1 成本模型的边界 62
8.2 代换成本 62
8.3 精确算术与累加器 62
8.4 Fact 存储与内存统计 63
8.5 Batch 与高层 theorem 的边界 63
8.6 与 VIPR 的可比实验 63

附录 A 修订提案、开放问题与冻结检查 65
A.1 提案登记 65
A.2 开放问题 65
A.3 冻结检查点 66
A.4 修订记录 66

第一章　数学模型与合约

本章定义验证器接受的数学对象，以及一次成功验证所证明的命题。RPA 的含义为 *Reformulation Proof Assembly*；RPX 的含义为 *Reformulation Proof eXecutable*。`.rpa` 是证明汇编源码，`.rpx` 是该源码的二进制证明证书。RPX 的执行对象是数学事实、解映射和合约义务；它不执行求解器搜索，也不修改输入模型。

每个方向要证的是一组**验证义务**（obligation）：kernel 根据该方向的 output model 和所请求合约生成这些待证检查项（见第1.7节与第1.8节），证书必须逐条证明完毕；方向结束时仍为 `PENDING` 的义务以 `E_OBLIGATION` 拒绝。每条义务由对象（输出变量的某个有限界或整数性、输出行的一个 side、目标 relation）和所需事实类型（`LE`、`EQ` 或 `INT`）唯一确定，初始状态为 `PENDING`，只有该方向 tape 中匹配的 `CHECK` 指令成功执行时才变为 `DONE`。义务集合由 kernel 决定，证书无法选择其子集，也无法为义务自声明表达式；待证表达式由 kernel 在检查时用当前 mapping 代换输出模型构造。

对证书而言，影响义务完成的途径只有三个：提供 mapping、提供从 input 静态事实和分支假设出发的推导链、以及在 `CHECK` 中提交事实引用。完成与否不取决于引用写法，而取决于提交事实的 $(\text{kind}, \text{expression})$ 是否与 kernel 构造的待证表达式精确相等；同一义务在不同分支可以对应不同的表达式。分支内已完成的状态在汇合时按可达分支取交集。义务的生成规则与表达式构造见第二章第2.5节，`CHECK` 指令族见第三章，完整示例见第七章。

1.1　规范范围与模型边界

v0.1 的数学域为具有有理数数据的普通混合整数线性规划（MILP）。本规范不直接赋予浮点数、非线性表达式、半连续变量、半整数变量、SOS、indicator constraint 或求解器专有对象任何语义。这些对象若需使用本规范，必须先转换为本章定义的 MILP，且转换本身必须有明确的可信边界。

`MCM` 在本手册中表示 *canonical mathematical model*，即规范化数学模型。本章规定其数学 schema；它不规定 MCM 的文件字节布局。**必须：** RPX 验证器必须针对调用者提供的、已经通过模型结构检查的规范化模型执行证明。证书中的模型摘要不得替代对模型结构的检查。

开放问题 O-01。 MCM 的文件格式、canonical serialization、SHA-256 输入字节以及 MPS 各种方言的导入规则尚未冻结。发布可互操作的 v0.1 工具链之前，必须另行规定它们。在此之前，不得声称两个独立的 MCM 序列化器必然产生相同摘要。

1.2　精确标量

数学标量属于 $\mathbb{Q}$，并表示为

$$a = \frac{p}{q}, \qquad p \in \mathbb{Z}, \quad q \in \mathbb{Z}_{>0}, \qquad \gcd(|p|, q) = 1.$$

零的唯一数值表示为 0/1。负号属于分子；分母不得为零或负数。表达式中的有限界、系数、目标常数和指令中的乘子均使用此数域。

必须： 任何影响证明接受与否的运算和比较必须遵循精确有理数语义。实现可以采用机器整数快路径、大整数提升和经过证明正确的数值过滤，但溢出、舍入或过滤不确定性不得改变数学结论。IEEE 浮点数近似相等和容差可行性不属于本规范。

RPA 十进制和科学计数法字面量表示十进制有理数，例如

$$\texttt{0.125} = \frac{1}{8}, \qquad \texttt{1e-6} = \frac{1}{1000000}.$$

模型导入器必须说明原始文件数字的解释方式。一个已经在外部系统中舍入的二进制浮点值，不能通过在证书中写出其十进制外观而恢复舍入前的模型。

1.3 规范化 MILP

一个模型写作

$$M = (V, R, f), \qquad V = (v_0, \ldots, v_{n-1}), \quad R = (r_0, \ldots, r_{m-1}).$$

变量和行的编号是从零开始的稳定整数编号。名称、注释、原始文件顺序以外的显示信息不参与任何数学推导。模型中的每个引用必须在对应的编号范围内。

1.3.1 变量

变量 v_i 定义一个实数坐标 x_i，并包含变量类型和有效上下界：

$$l_i \le x_i \le u_i, \qquad l_i \in \mathbb{Q} \cup \{-\infty\}, \quad u_i \in \mathbb{Q} \cup \{+\infty\}.$$

缺失的下界表示 $-\infty$，缺失的上界表示 $+\infty$。无穷大是缺失界的标记，不是可参与有理数运算的标量。每个变量的类型为以下一种：

类型	附加数学条件
`CONTINUOUS`	无附加整数条件，$x_i \in \mathbb{R}$。
`INTEGER`	$x_i \in \mathbb{Z}$。
`BINARY`	$x_i \in \mathbb{Z}$ 且 $0 \le x_i \le 1$。

必须： BINARY 的隐含 0 下界、1 上界和整数性必须同时作为模型事实和输出验证义务的一部分。若模型还提供额外二进制变量界，规范化阶段取交集，得到 $l_i = \max(0, l_i^{\text{explicit}})$ 和 $u_i = \min(1, u_i^{\text{explicit}})$。模型 schema 要求有效界满足 $l_i \le u_i$。本草案把违反此结构要求的模型视为无效模型；若需表示不可行模型，可以通过合法的行与变量条件表示其矛盾。

`split.bin` 的前提是模型类型显式为 `BINARY`。整数变量即使声明界为 $[0, 1]$，也不能自动作为该指令的操作数；它仍可使用整数分裂指令。

1.3.2 约束行

行 r_j 的线性部分为

$$a_j(x) = \sum_{i \in I_j} a_{ji} x_i, \qquad a_{ji} \in \mathbb{Q} \setminus \{0\}.$$

该行规定

$$L_j \le a_j(x) \le U_j, \qquad L_j \in \mathbb{Q} \cup \{-\infty\}, \quad U_j \in \mathbb{Q} \cup \{+\infty\}, \quad L_j \le U_j.$$

没有有限界的行不产生可行性义务。当 $L_j = U_j = b_j \in \mathbb{Q}$ 时，该行为等式行；否则为范围行。单侧约束是范围行的特殊情形。常数行合法，并可用于表达模型不可行性。

行的常数项统一移入界。例如 $2x + 3 \le 7$ 的模型行可表示为 $2x \le 4$。RPX fact 的表达式则保留常数项，不再单独存储右端项。模型行和目标函数的非零系数列表同样必须按变量编号严格升序，合并重复变量并删除零项；其有限标量遵循规范化有理数规则。

1.3.3 目标函数

规范化目标函数为

$$f_M(x) = c_M + \sum_i d_{Mi} x_i, \qquad c_M, d_{Mi} \in \mathbb{Q},$$

且目标方向统一为最小化。原始最小化目标保持原值，原始最大化目标整体乘以 -1，包括目标常数。原始方向和展示名称可以保留为外部元数据。

修订提案 P-01。v0.1 统一使用最小化目标，并要求调用者明确要求的方向、合约 profile 和 objective relation。证书声明必须与调用者请求一致；不得由证书自行降低目标关系或省略一个请求的方向。

这意味着目标偏移是证明的一部分。删除或修改 c_M 会改变目标关系，即使所有变量系数保持相同。对于原始最大化问题，`LE` 关系在规范化目标上表示不增加，在原始目标上对应不降低。

1.3.4 可行域与扩展最优值

模型的可行域定义为

$$\mathcal{F}(M) = \{x \in \mathbb{R}^n : \text{所有变量 domain 和所有约束行成立}\}.$$

规范化最优值定义为扩展实数

$$\mathrm{val}(M) = \inf_{x \in \mathcal{F}(M)} f_M(x) \in \mathbb{R} \cup \{-\infty, +\infty\},$$

其中 $\inf \varnothing = +\infty$。非空可行域上的目标无下界时，最优值为 $-\infty$。本定义不要求最优解存在。

1.4 规范化仿射表达式

仿射表达式写作

$$e(x) = c + \sum_{k=0}^{t-1} a_k x_{i_k}.$$

其规范形式满足：

1. $0 \le i_0 < i_1 < \cdots < i_{t-1} < n$；
2. 每个 a_k 为非零的规范化有理数；
3. 常数 c 为规范化有理数；
4. 同变量项已经合并，零系数项已经删除。

空 term 列表表示常数表达式；常数为零时表示零表达式。forward 和 reverse 各有独立的表达式池；池由有限个 entry 组成，每个 entry 承载一个上述规范形式的表达式，由 `ExprId` 标识，变量编号均指向该方向的 input model（编码见第四章）。

两个规范化表达式的相等是常数及各变量系数的精确数值相等。不同 `ExprId` 可以指向数值相同的表达式。不同 `ScalarRef` 也可能解码为同一个有理数。**必须：** 未经验证的编号相等、对象地址相等或散列值相等不得替代精确表达式比较。实现可以在经过验证的 interning 或精确比较结果上建立缓存。

规范化不包含按比例约简、改符号或寻找逻辑等价命题。例如 $\mathrm{LE}(x-1)$ 与 $\mathrm{LE}(2x-2)$ 是等价不等式，但在本 ISA 中是不同 proposition。$\mathrm{EQ}(e)$ 与 $\mathrm{EQ}(-e)$ 同理。若指令需要特定表达式，证明必须通过显式线性组合得到该表达式。

1.5 事实类型

事实（fact）由类型、表达式和有效作用域组成：

$$\text{Fact} = (\text{kind}, \text{expr}, \text{scope}).$$

核心类型如下。

类型	数学含义	示例
`LE`	$e(x) \le 0$	$\text{LE}(3x_4 - 2x_9 - 7)$。
`EQ`	$e(x) = 0$	$\text{EQ}(x_3 + x_7 - 1)$。
`INT`	$e(x) \in \mathbb{Z}$	$\text{INT}(2x_3 - x_8 + 5)$。

fact 的 proposition 是二元组 $(\text{kind}, \text{expr})$。scope 限定该 proposition 在哪些分支条件下成立，但不参与 branch yield 的命题相等比较。一个 fact 不携带独立 RHS、浮点容差、求解器 basis 或可执行代码。

1.6 输入模型的静态事实

以下引用直接从当前方向的 input model 获得事实，无需 `LOAD` 指令。这里 a_j 为行线性部分，x_i 为输入变量。

RPA 引用	FactKind	表达式与前提
`@in.r[j].hi`	`LE`	$a_j(x) - U_j$；要求有限上界。
`@in.r[j].lo`	`LE`	$-a_j(x) + L_j$；要求有限下界。
`@in.r[j].eq`	`EQ`	$a_j(x) - b_j$；要求等式行 $L_j = U_j = b_j$。
`@in.v[i].hi`	`LE`	$x_i - u_i$；要求有限有效上界。
`@in.v[i].lo`	`LE`	$-x_i + l_i$；要求有限有效下界。
`@in.v[i].int`	`INT`	x_i；要求 `INTEGER` 或 `BINARY`。

等式行仍具有有限 hi 和 lo 事实；输出 obligation 则按等式行规则生成一个 EQ 义务。连续变量不能提供 `.int` 事实，即使其有限上下界相同。引用不存在的有限 side、错误类型的模型实体或越界编号必须拒绝。所有静态模型事实在该方向的任意 scope 中有效。分支假设 `@case` 属于另一类静态引用，见第二章。

1.7 两个执行方向

一次 reformulation 验证绑定两个模型 M_S 和 M_T。每个启用的方向执行独立证明：

方向	Input	Output	要证明的解映射
`FORWARD`	M_S	M_T	$F : \mathcal{F}(M_S) \to \mathcal{F}(M_T)$。
`REVERSE`	M_T	M_S	$G : \mathcal{F}(M_T) \to \mathcal{F}(M_S)$。

fact register、表达式池、mapping、scope 栈和 obligation bitmap 均为 direction-local。反向执行不得引用正向 fact 或依赖正向 mapping。两方向共享证书的标量池和绑定的模型，但不共享证明状态。

一个方向的 mapping 必须在其每个可行输入上为每个输出变量提供唯一选定的值。mapping 可以是单个仿射映射，也可以由有限的结构化分支表示。验证器证明每个可达分支的输出都满足 output model；它不需要在验证时构造实际可行点。重叠分支的结果可以不同，只要分别满足全部要求。若需要输出可执行的点变换，必须同时规定确定的分支选择规则。

1.8 目标关系与合约 profile

对一个方向，设 input model 为 M_I、output model 为 M_O、mapping 为 E。目标差为

$$D_E(x) = f_{M_O}(E(x)) - f_{M_I}(x).$$

每个方向选择一种 objective relation：

关系	要求	验证义务
`NONE`	无目标关系要求	不生成 objective obligation。
`EQ`	$D_E(x) = 0$	一个目标 EQ obligation。
`LE`	$D_E(x) \le 0$	一个目标 LE obligation。

目标关系只在 input model 的可行域上要求成立。不论 relation 为何，每个启用方向都必须完成全部 output mapping、变量 domain 和行可行性义务。`NONE` 不允许略过模型可行性检查。

1.8.1 方向性命题（资料性）

数学上可以只要求一个独立方向。例如一个 forward 证明意味着每个 source 可行点都有满足指定目标关系的 target 可行点。它不能单独声称两个模型可行性或最优值等价。本草案基础格式不编码这种单方向合约；它是未来扩展可采用的数学命题。当前合法 RPX 必须同时启用两个方向，并选择下面两个等价 profile 中的恰好一个。

1.8.2 可行性等价

`feasibility-equivalent` profile 要求 forward 和 reverse 同时启用。objective relation 可以分别为 `NONE`、`EQ` 或 `LE`。成功验证证明

$$\mathcal{F}(M_S) = \varnothing \quad \Longleftrightarrow \quad \mathcal{F}(M_T) = \varnothing.$$

它还证明所给的两个可行解转换存在；这不意味着两个不同维度可行域逐点相同。若两个方向均选择 `NONE`，不声称目标值等价。

1.8.3 最优值等价

`opt-equivalent` profile 要求 forward 和 reverse 同时启用，且每个方向的 objective relation 必须为 `EQ` 或 `LE`。由两个方向分别得到

$$f_T(F(x)) \le f_S(x), \qquad f_S(G(y)) \le f_T(y),$$

其中 EQ 是 LE 的特殊情形。对两个不等式分别取下确界，得到

$$\boxed{\mathrm{val}(M_S) = \mathrm{val}(M_T)}.$$

该结论适用于空可行域和目标无下界的情形。映射同时保证两个可行域非空性一致，因而不会把不可行性与目标无下界混淆。若两者具有有限最优值，任一侧的最优可行点通过相应映射成为另一侧的最优可行点。

必须： 最优值等价不要求 $G \circ F$ 或 $F \circ G$ 为恒等映射。变量删除、冗余变量引入和支配关系变换通常没有双射解映射。`opt-equivalent` 也不保证每个次优解的目标值完全保持不变；该更强要求应明确选择两个方向的 `EQ`。

1.8.4 合约标志的约束

二进制 contract flags 的字段布局见二进制格式章。本章规定它们的语义有效性：

1. forward 和 reverse 都必须启用；
2. 两个等价 profile 必须恰好声明一个，不得同时声明或同时省略；
3. optimum profile 要求两个方向的 objective relation 均非 `NONE`；
4. reserved relation 和 reserved flag 位必须为零，或按格式章规定拒绝。

调用者请求与证书声明不一致属于 `E_CONTRACT`，不允许通过读取证书后自动调整请求来消除该错误。

1.9 摘要绑定与接受结果的含义

RPX header 包含 source 和 target 的 SHA-256 摘要。验证器必须比较它们与调用者提供的两个 canonical model 的摘要；方向交换不能交换 header 的 source/target 字段含义。模型摘要不匹配必须拒绝为 `E_HASH`。

摘要只有在明确的 canonical serialization 约定下才有可互操作意义。调用者、MCM 导入器和验证器必须采用同一约定，并明确哪些外部导入步骤属于可信计算基。RPX 成功证明的是绑定后两个数学模型之间的合约。它不自动证明外部 MPS 文件与 MCM 完全一致，也不证明模型名、原始浮点缓冲区或求解器中隐藏的数据与已绑定模型相同。

完整接受结果应至少报告：绑定的两个摘要、合约 profile、启用方向和各方向的目标关系。只有所有启用方向成功，整个 RPX 才能报告 `VERIFIED`。

第二章　执行模型、作用域与验证义务

本章规定 RPX proof VM 的抽象状态和控制语义。具体字段布局与各指令的 payload 见相应章节；这里的状态是数学规范，不要求实现采用相同的内存结构。验证器可以顺序解释经检查的字节视图，无需构建证明 DAG 或执行 proof search。

2.1　方向状态

每个启用方向的初始状态包含：

$$\Sigma = (M_I, M_O, \mathcal{E}, \mathcal{F}, \mathcal{S}, \mathcal{M}, \mathcal{O}, \nu, \mathrm{pc}).$$

各分量如下。

分量	含义
M_I, M_O	已绑定且通过结构检查的 input、output 模型。
$\mathcal{E}$	该方向的只读规范化表达式池。
$\mathcal{F}$	当前仍可引用的已验证派生 fact。
$\mathcal{S}$	当前 scope 栈及各层 branch assumption。
$\mathcal{M}$	output 变量映射状态。
$\mathcal{O}$	由模型和合约生成的 obligation 状态。
ν	下一个隐式派生 FactId 的语法编号。
pc	当前 tape 或 branch byte range 中的指令位置。

根 scope 不带 branch assumption。静态映射 section 在执行 tape 之前初始化 $\mathcal{M}$。每个方向的 FactId 从零重新开始。forward 完成后必须清除方向局部的可变状态，再初始化 reverse。

2.2　事实寄存器与隐式编号

RPA 中的 `%name` 是汇编器符号；RPX 中的派生引用使用 28-bit FactId。每个派生 fact 只定义一次。`DROP` 不允许重用已经使用或保留的逻辑编号。

定义 fact 的普通指令按 tape 语法顺序占用一个 FactId。分裂指令的编号规则如下：

1. 先按编码顺序完整遍历各 child block；
2. nested split 同样采用深度优先的 child-first 顺序；
3. 所有 child block 的语法定义占号之后，才为本次 split 的 k 个 merged results 连续分配 FactId；
4. 即使 child 或 parent 已被证明不可达，其语法中存在的定义仍占号，但不提交 live fact。

修订提案 P-03。派生 FactId 采用 direction-wide、单调递增的语法编号，split 输出在 child block 之后编号；`DROP` 仅能删除当前 scope 定义的 fact。这固定了无显式 destination 字段的编码行为，并使 branch rollback 不受祖先 fact 删除影响。

例如，根 scope 已定义 FactId 0，二分支 split 的 child 0 定义 1 和 2，child 1 定义 3，而 split 有两个 outputs，则 merged facts 编号为 4 和 5。child 1 不能引用 1 或 2；它们已经结束作用域。split 后不能引用 1、2 或 3，只能引用仍 live 的祖先 fact 和 merged facts。

必须：编号不足以证明引用有效。每个派生引用还必须通过定义先后、live 状态和 scope 检查。禁止 future reference、重复定义和超过 28-bit 编号空间。语法遍历必须采用检查后的长度和 checked arithmetic；整数回绕不得改变占号结果。

2.3 作用域和 FactRef 解析

scope 的逻辑身份是从根到当前分支的路径。物理实现可以使用 scope ID 或栈深度，但深度相同的 sibling scopes 不相同。当前指令只可以引用：

1. input model 的有效静态事实；
2. 当前 scope 或其祖先 scope 中仍 live 的派生 fact；
3. 当前 scope 或其祖先 split 对应的 branch assumption。

sibling scope、已经退出的 child scope 和已删除 fact 不可引用。scope 的逻辑身份是从根到当前分支的完整路径，物理深度相同不构成同一个 scope；两个 sibling 互不可见。scope 出口释放所有该 scope 的 local facts。若有 yield，必须先按顺序保存其 proposition，再释放 local facts：保存的是 (kind, expression) 的值，而不是 child 的 FactId；释放后该 FactId 不再可用，但保存的值足以完成汇合比较。

数学上，表达式可以通过只读 `ExprView` 提供，来源包括：

- 方向表达式池 entry（定义见第一章，编码见第四章）；
- canonical input model 的有限 side，例如 `@in.v[i].hi` 对应的 $x_i - u_i$；
- kernel 根据 split 操作数构造的 assumption view，例如 `@case`。

因此 branch yield 可以直接引用静态 fact，不必先派生一个等值的本地 fact；合并按 proposition 精确相等进行，不要求为其表达式在表达式池中搜索或插入一个相同 entry。**必须：**保存的 proposition 不得持有已经释放的 scope 状态；它的表达式数据必须继续有效，且所有相等、符号和常数比较仍使用精确数值语义。来源不同的表达式按同一判据比较：以解码后的精确有理数表达式逐项相等为准，不得以 ExprId、句柄或对象地址相等代替。派生算术指令声明的输出表达式仍必须来自相应方向的 expression pool。内部 live-fact record 是否只保存一个 `ExprId` 属于实现选择，而非 ISA 要求。

示例沿用上例的编号：root 已定义 `%0`，child 0 定义 `%1`、`%2`，child 1 定义 `%3`，merged facts 为 `%4`、`%5`。

```
%0 = lin.le $s { +1 @in.r[17].hi }               ; LE(s), s = a17(x) - U17
(%4, %5) = split.bin @in.v[0] {
case 0:
    %1 = lin.le $s { +1 %0 }                     ; 命题 LE(s)
    %2 = lin.le $t { +1 @in.r[17].hi -1 @case } ; @case 是 EQ; 不参与 yield
    yield %1, @in.v[5].hi
case 1:
    ; child 0 已退出，引用 %1 或 %2 以 E_REF 拒绝
    %3 = lin.le $s { +1 @in.r[17].hi }           ; 同一命题，不同推导路径
    yield %3, @in.v[5].hi
}
; 汇合：位置 0 都是 LE(s) → %4；位置 1 都是 LE(x5 - u5) → %5
```

两支通过不同路径提交同一命题 LE(s)：child 0 经祖先 fact `%0`，child 1 直接经静态事实 `@in.r[17].hi`；汇合只比较保存的值，不比较引用编号或推导路径。`%2` 不参与 yield，随 child 0 退出释放；`%1`、`%2`、`%3` 在汇合后失去可见性。位置 1 直接 yield 静态事实 `@in.v[5].hi`，其表达式 $x_5 - u_5$ 由

input model 的有限 side 提供，不要求在表达式池中有对应 entry。

2.3.1 分支假设

FactRef tag `0x7` 的 index 是相对当前 scope 的 assumption 深度。`@case` 与 `@case[0]` 相同，表示最内层 split 的当前假设；`@case[1]` 表示它的父层假设。根 scope 没有 `@case`。越过现有 assumption 栈的引用属于 `E_REF`。

分支假设由 kernel 构造，不由证书任意声明：

Split	Branch	假设 proposition
`split.bin` x_i	0 / 1	$\mathrm{EQ}(x_i)$ / $\mathrm{EQ}(x_i - 1)$。
`split.int.thr` e, t	le / ge	$\mathrm{LE}(e - t)$ / $\mathrm{LE}(-e + t + 1)$。
`split.int.range` e, L, U	$k \in [L, U]$	$\mathrm{EQ}(e - k)$。
`split.sign` e	le / ge	$\mathrm{LE}(e)$ / $\mathrm{LE}(-e)$。

整数分裂必须同时具备对应的 INT 前提，range 分裂还必须验证范围上下界。分支集合必须完整覆盖该 scope 中可能的 input points。sign 分裂在 $e = 0$ 时重叠；重叠不会影响覆盖证明或验证正确性。

修订提案 P-06。若工具链需要把证书用作实际点变换程序，按编码顺序选择第一个假设成立的分支；因此 `split.sign` 在零点选择 `le`。纯 proof verification 仍必须检查两个分支。

2.3.2 删除事实

`DROP` 的每个目标必须是当前 scope 中仍 live 的派生 fact。不允许删除模型静态事实、branch assumption、祖先 fact 或已经 dead 的 fact。删除后它不得再被引用或 yield。无论是否释放了物理存储，原 FactId 永远不能再次定义。祖先 fact 可在返回祖先 scope 后删除。

这允许编译器根据 liveness 插入删除操作，同时保持 branch state 的可回滚性。实现若以总历史 FactId 为索引保留一个永久数组，`DROP` 本身并不会获得 $O(\text{live facts})$ 的内存占用；此复杂度需要可回收存储和适当的 live 索引结构。

2.4 输出映射状态

每个 output variable 的 mapping 处于以下三态之一：

状态	含义
`UNASSIGNED`	当前路径尚未定义该变量；初始化时由 `DEFER` 明确声明。
`KNOWN(e)`	已定义为该方向 input variables 上的规范化仿射表达式 e。
`SEALED_PIECEWISE`	已在所有可达分支定义，且所有依赖它的验证义务已经完成；不可再进行涉及该变量的代换。

修订提案 P-02。mapping merge 使用上述三态。`SEALED_PIECEWISE` 保留“输出变量在每个可达路径已经定义”的证明信息，允许 `RET` 接受 piecewise map，同时避免在 merge 后构建 piecewise expression AST。若可达 children 中已定义和未定义混合则立即拒绝；只有全部未定义才能继续保持 `UNASSIGNED`，以避免引入第四种“部分路径已定义”状态。

静态 map section 必须覆盖每个 output variable 恰好一次，使用 COPY_RANGE、COPY_PAIRS、CONST、AFF 或 DEFER。copy、const 和 aff 初始化为 KNOWN；defer 初始化为 UNASSIGNED。静态映射 section 只能使用该方向 input variables 和 expression pool。

map.copy、map.const 和 map.aff 只能把一个 UNASSIGNED 的 deferred variable 转为 KNOWN。**必须：**映射在每条可达路径上只允许赋值一次。禁止覆盖 KNOWN 或 SEALED_PIECEWISE。mapping 本身不产生 LE、EQ 或 INT fact，也不完成任何 obligation。

2.5 验证义务的生成

obligation 集合必须由 kernel 根据 output model 和所请求合约生成，不能由证书选择其子集。每个 obligation 的状态为 PENDING 或 DONE。该方向初始状态中所有义务为 PENDING。

对象	必须生成的 obligation
Output variable	每个有限有效下界和上界分别一个 LE 义务；INTEGER、BINARY 各有一个 INT 义务。binary 的隐含界必须包含在内。
Output equality row	一个 EQ 义务；不另生成该行的 hi、lo 义务。
Output ranged row	每个有限 side 分别一个 LE 义务。
Objective	relation 为 EQ 或 LE 时生成相应的一个义务；NONE 不生成。

mapping 完整性另行检查，不能因为某个 output variable 没有任何有限界、不参与约束或目标而略过其定义。

2.5.1 义务的表达式

设 $E_i(x)$ 为 output variable y_i 的当前 KNOWN mapping。kernel 自己构造以下规范化表达式：

$$
\begin{aligned}
\text{row.hi}(j): &\quad \sum_i a_{ji}E_i(x) - U_j,\\
\text{row.lo}(j): &\quad -\sum_i a_{ji}E_i(x) + L_j,\\
\text{row.eq}(j): &\quad \sum_i a_{ji}E_i(x) - b_j,\\
\text{var.hi}(i): &\quad E_i(x) - u_i,\\
\text{var.lo}(i): &\quad -E_i(x) + l_i,\\
\text{var.int}(i): &\quad E_i(x),\\
\text{obj}: &\quad f_{M_O}(E(x)) - f_{M_I}(x).
\end{aligned}
$$

所有非零 support 中的 output variables 必须处于 KNOWN。涉及 UNASSIGNED 或 SEALED_PIECEWISE 的代换必须拒绝。常数行和常数目标差不依赖任何 output variable。

fact 形式的 CHECK 必须精确匹配该 CHECK 所需 FactKind 和上述表达式。以一个语义更强但表达式不同的不等式代替所需表达式不合法；应先通过 weaken.le 或显式线性组合得到目标表达式。目标检查允许一种明确的类型加强：CHECK.OBJ.EQ 提供精确目标差的 EQ 证明，可以完成所请求的 EQ 或 LE objective obligation；CHECK.OBJ.LE 只能完成 LE。relation 为 NONE 时两者都没有可完成的 objective obligation。

2.5.2 EXACT 检查模式

EXACT 是 CHECK 的 kernel 检查模式，不是相信 assembler 的承诺。它不分配派生 FactId。kernel 在完成代换后，仅接受以下无需额外 fact 的充分条件：

1. EQ 类 CHECK 的表达式精确为零；
2. LE 类 CHECK 的表达式没有变量项，且常数 $c \le 0$；
3. INT 类 CHECK 的常数和所有系数均为整数，且每个非零项引用 input model 中声明为 INTEGER 或 BINARY 的变量。

INT 的 EXACT 检查不使用行等式推导连续变量的整数性。更复杂的整数性必须以显式 INT fact 证明。EXACT 条件由 CHECK 操作码确定；例如 CHECK.OBJ.EQ 即使完成 LE obligation，其 EXACT 模式仍要求目标差精确为零。模式位和 payload 编码由指令格式规定。

2.5.3 完成义务

一个成功 CHECK 只把它所对应的一个 PENDING obligation 置为 DONE。对不存在、错误 side、错误 objective relation 或已经 DONE 的义务执行 CHECK，必须拒绝为 E_OBLIGATION。同一父状态在不同 branches 中各自完成同一义务不算重复检查，因为它们是独立子状态。在 split 之前已经 DONE 的义务则被各子状态继承，不得重复检查。

每次 CHECK 的依赖集合由 output model 的规范化非零 support 确定。行义务依赖该行的 output variables；目标义务依赖目标非零系数中的 output variables；变量界和整数性义务依赖自身变量。该集合是 mapping merge 时判定是否允许封存变量的依据。

2.6 结构化分裂与状态事务

RPX 没有 JMP 或非结构化回边。split 指令通过其 payload 中的 byte ranges 包含有限个完整 child blocks。每个 child 从同一个 parent 状态开始；它继承 parent live facts、mapping 和 obligation 状态，并增加一个由 kernel 产生的 assumption。



必须： child 0 的检查结果、局部 mapping 或局部 fact 不得作为 child 1 的初始状态。实现可以使用 snapshot、copy-on-write 或 transaction log，但可观察行为必须等价。退出 child 时回滚它的局部状态，再使用保存的结果计算 parent merge。

2.6.1 Block 终结规则

每个 child block 必须以 `YIELD` 或 `CLOSE.FALSE` 终结。顶层 tape 必须以 `RET` 或 `CLOSE.FALSE` 终结。终结指令后不得存在该 block 的其他指令；尾部仅允许格式规定的 alignment padding。child 内不得使用 `RET`，顶层不得使用 `YIELD`。open split 内的终结只结束当前 child，不直接结束整个方向。

split 的 branch 数量和范围必须与操作码要求一致。integer range 的 branch k 必须对每个整数 $k \in [L, U]$ 恰好出现一次。branch byte ranges 必须各自通过完整结构检查。

2.6.2 Yield 与 proposition merge

`YIELD` 提供有序的 k 个 FactRef，数量必须与 enclosing split 声明相同。每个引用在 yield 时必须有效且 live。yield 保存的是 $(\text{kind}, \text{canonical expression})$，不是 child FactId。

对每个 yield 位置 j，所有可达 children 必须提供同一 proposition：

$$(K_{b,j}, e_{b,j}) = (K_j, e_j) \quad \text{对每个可达 child } b.$$

相等要求 FactKind 相同、解码后的表达式精确相等；不接受按比例相等。kernel 为每个位置提交一个 parent-scope fact，其编号遵循第2.2节。该 fact 的数学表达式仍属于当前方向的 input variables。

若只有一个可达 child，它的 yield tuple 成为 merge result。若所有 children 不可达，则 parent 也不可达，不提交 live merged facts，但仍为声明的 k 个结果保留语法 FactId。不可达 child 不必提供 yield tuple。

2.6.3 义务汇合：交集

令 B 为可达 child 集合，每个 child 的 DONE 集合为 D_b。非空 B 的 parent DONE 集合为

$$D_{\text{parent}} = \bigcap_{b \in B} D_b.$$

bitmap 实现必须逐位 AND，不得使用 OR。不可达 child 不参与此交集。当 B 为空时，parent 标记不可达，剩余义务在该 scope 中不再有数学要求。

例如一个 row obligation 在 child 0 为 DONE、在 child 1 为 PENDING，两支都可达，则 merge 后它仍为 PENDING。不能用“至少一个分支证明过该行”替代“每个可达分支证明该行”。

Child 0	Child 1	Parent
DONE	DONE	DONE
DONE	PENDING	PENDING
PENDING	DONE	PENDING
DONE	UNREACHABLE	DONE
UNREACHABLE	UNREACHABLE	UNREACHABLE

2.6.4 映射汇合与封存

对非空可达 child 集合，kernel 逐 output variable 处理：

1. 各 child 都有精确相同的 `KNOWN(e)`，则保留 `KNOWN(e)`；
2. 各 child 都为 `UNASSIGNED`，则保留 `UNASSIGNED`；
3. 各 child 都已定义，但表达式不同，或至少一个 child 已经 `SEALED_PIECEWISE`，则仅当所有依赖该变量的 obligations 在每个可达 child 中均为 DONE 时，产生 `SEALED_PIECEWISE`；否则拒绝；
4. 若已定义和未定义的可达 child 混合，按 mapping 完整性规则拒绝，防止 merge 后赋值覆盖已定义路径。

一个不可达 child 不影响 mapping 完整性。当所有 children 不可达时，无需进行 mapping merge。

封存必须检查每个相关 obligation，包括尚未检查的目标义务。若目标依赖该变量，不能只完成其变量界和约束行后封存。mapping 封存后不得重新赋值，也不得再参与新的 substitution。

下面为 piecewise mapping 的示意；`...` 是说明性省略，不是 RPA 指令。

```
() = split.sign $e {
case le:
    map.aff @out.v[7], $neg_e
    ... ; check every obligation depending on output variable 7
    yield
case ge:
    map.aff @out.v[7], $e
    ... ; check the same obligations in this branch
    yield
}
ret
```

merge 后变量 7 可以处于 `SEALED_PIECEWISE` 并通过 `RET`。若任意可达 child 留下依赖变量 7 的 PENDING obligation，split 必须拒绝。即使在 merge 后 “打算再证明” 该义务，也不允许丢弃它所需的不同 branch mappings。

2.7 不可达作用域

`CLOSE.FALSE` 必须引用当前 scope 可见的矛盾 fact：

$$\mathrm{LE}(c),\quad c > 0, \qquad \text{或} \qquad \mathrm{EQ}(c),\quad c \neq 0,$$

且表达式没有变量项。INT fact 或含变量表达式不能直接关闭 scope。成功后，该 scope 标记为 `UNREACHABLE`。这表示没有 input feasible point 同时满足该 scope 的路径假设。

根 scope 不可达证明 input model 的可行域为空。对应方向的可行映射与目标关系因此真空成立，不要求 mapping 或 obligation 完成。其他启用方向仍必须验证。尤其在双向合约中，不能由 source 不可行直接推断 target 不可行。

2.7.1 不可达代码的结构处理

如果 nested split 的所有 children 都不可达，当前 parent 会在其 terminal 之前变为不可达。实现可以跳过后续证明运算、mapping 更新和 CHECK 的数学效果，但必须继续执行结构性遍历，直到该 block 合法终结。

必须： 不可达性不得掩盖格式错误、未知 opcode、错误 payload 长度、保留位、非法 byte range、缺失 terminal 或 FactId 编号空间溢出。所有表达式和标量 pool 都必须通过结构与规范化检查，无论执行

时是否实际访问其 entry。证书语法中的定义和 scope 仍要记账，保留相应 FactId。已经不可达的 scope 不提交新事实，也不要求其后续数学推导再次证明正确。跳过数学效果不允许跳过未知 extension 的支持检查。

2.8 RET 与方向接受条件

正常执行到顶层 `RET` 时，kernel 必须检查：

1. 当前为根 scope，没有未结束的 split；
2. 所有 output variables 均为 `KNOWN` 或 `SEALED_PIECEWISE`；
3. 所有由模型和合约生成的 obligations 均为 DONE；
4. 顶层 tape 的结构与终结位置合法。

任一 `UNASSIGNED` variable 或 PENDING obligation 都必须拒绝。根 scope 已经不可达时，合法顶层 terminal 结束该方向的真空证明。只有所有启用方向都满足各自接受条件，证书才被接受。

2.9 错误类别与资源结果

以下名称是诊断类别，本草案尚未冻结数值 error ABI。实现应报告方向和 tape offset，必要时附带 model ID、FactId 或 section 信息，但不得在错误之后继续提交部分证明结果。

类别	典型条件
`E_FORMAT`	长度、alignment、padding、section 或 terminal 结构错误。
`E_VERSION`	不支持的版本或字节序。
`E_UNSUPPORTED`	未支持 opcode、extension 或 theorem；不可静默忽略。
`E_HASH`	source 或 target 摘要不匹配。
`E_CONTRACT`	合约标志无效、缺方向或与调用者要求不一致。
`E_SCALAR`	非规范化有理数、非法标量池 entry 或标量表示。
`E_EXPR`	非规范化表达式。
`E_REF`	非法编号或 FactRef、future/dead/sibling/已退出 scope 引用、缺失 input side 或越界 assumption 深度。
`E_KIND`	指令或 CHECK 引用的 FactKind 不符。
`E_SCOPE`	删除祖先 fact，或在错误的 scope 使用 RET/YIELD 等终结指令。
`E_BRANCH`	分支不完整、yield 数量错误或 yield proposition 不同。
`E_PROOF`	线性组合、取整、整数性、BOX 或矛盾检查不成立。
`E_MAP`	重复赋值、mapping 不完整、代换未定义变量或非法 merge。
`E_OBLIGATION`	重复或不存在的 CHECK、未完成义务、目标关系类型不符。

API 应区分以下最终状态：

状态	含义
`VERIFIED`	所请求合约的全部启用方向已被证明。
`INVALID`	已检测到违反格式、引用、数学推导或义务条件的证书。
`UNSUPPORTED`	验证器不支持其版本、mandatory extension 或 theorem。
`RESOURCE_LIMIT`	达到调用者配置的时间、内存、深度或算术资源限制；未得出接受结论。
`IO_ERROR`	模型或证书数据无法可靠读取，或读取期间完整性条件失效。

`RESOURCE_LIMIT` 不表示数学证明错误，不能作为 `VERIFIED`。实现可声明限制，包括 split nesting、term count、bigint 位数、expression expansion、live facts、总指令数和总算术工作量。格式字段自身限制与实现资源限制应分别报告；例如 16-bit aux 和 28-bit FactRef 是格式上限，不是保证能够在可用 RAM 内完成验证的承诺。

2.10 正确性不变式

令 $P_s(x)$ 为 input model 可行性和 scope s 的所有祖先路径假设的合取。执行到任意有效状态时，必须保持：

1. 每个可见 live fact 的 proposition 对所有满足 P_s 的点成立；
2. 每个 `KNOWN` mapping 表达式在当前路径上为相应 output variable 的定义；
3. 每个 DONE obligation 对满足 P_s 的点及当前路径映射成立；
4. 每个 `SEALED_PIECEWISE` variable 在全部已合并可达路径上有定义，且所有依赖它的 obligations 已被证明；
5. `UNREACHABLE` scope 满足 $\{x : P_s(x)\} = \varnothing$。

静态模型事实建立初始 fact 不变式。每条算术指令由其局部精确检查保持第 1 项；映射只定义值，CHECK 将已成立命题与 output obligation 的表达式匹配。结构化 split 的穷尽性保证每个满足 parent 条件的点属于至少一个 child。yield proposition 相等由此允许移除 child assumption；DONE 交集保证义务在所有可达 children 中成立。mapping 封存条件保证丢弃不同仿射表达式后不会留下需要它们的未证明义务。矛盾 fact 则建立不可达性。

对 tape 的结构作归纳，以上不变式覆盖 nested split。`RET` 的完整性检查将局部不变式提升为该方向的可行解映射和目标关系。两个方向独立接受后，按第1.8节推出对应合约结论。此为 soundness proof obligation 的概要；实现审计或形式化证明必须同时覆盖字节解码、精确算术、模型绑定和状态 rollback。

2.11 执行成本与存储说明（资料性）

顺序 tape 避免为证明推导建立 DAG 和进行拓扑排序。它不消除稀疏代换、exact arithmetic 或支配性资源成本。一条行义务的代换可能访问

$$\sum_{i \in \mathrm{supp}(a_j)} \mathrm{nnz}(E_i)$$

个映射项；这些项的合并还可能产生比原行更大的表达式。线性组合的乘法、加法和 gcd 成本取决于精确整数的位数，不能一律视为常数时间。

可变内存包含 live facts、mapping、obligation bitmap、scope transaction records 和稀疏算术工作区。read-only mmap 使文件字节可被按需访问，但不表示验证没有 page fault 或模型散列成本。八字节 fact record 是可能的实现布局，不是整个 VM 的内存复杂度证明。任何性能声明都应说明证书尺寸、访问的 sparse terms、标量位数、模型读取成本和是否包含完整双向验证。

第三章　核心指令集参考

本章规定 RPA/RPX 草案版本 0.1 的 28 条核心指令。每个条目同时给出文本汇编形式、二进制载荷、前置条件、数学推导规则、状态变化与拒绝条件。指令集作用于当前执行方向的输入模型与输出模型；正向和反向使用相同操作码，不共享寄存器、映射或义务状态。

3.1　公共编码与执行约定

3.1.1　指令头与字节序

所有指令采用以下 8 字节头。多字节字段均以 little-endian 编码；字段没有 C ABI 隐式填充。

```
InstHeader {
    opcode:        u8;
    flags:         u8;
    aux:           u16;
    payload_bytes: u32;
}
```

指令地址必须 8 字节对齐。令当前地址为 p，则本指令占用的完整区间为

$$[p, \mathrm{align}_8(p + 8 + \mathrm{payload_bytes})).$$

载荷结束与下条指令之间的填充字节必须为零，并且不计入 `payload_bytes`。本章列出的载荷长度为精确长度；额外字段、截断字段或未解释的尾字节均以 `E_FORMAT` 拒绝。所有未使用的 `flags`、`aux` 与显式 `reserved` 字段必须为零。对长度、计数、偏移的检查必须使用无溢出的算术。

3.1.2　操作数与寄存器定义

`ExprId`、`FactRef`、`ScalarRef`、输入及输出对象索引都是 32 位字段。表达式编号仅在当前方向的表达式池中有效。`FactRef` 可以指向活跃派生事实、输入模型声明的事实，或当前可见分支假设；取值规则见机器模型章。

事实产生指令没有显式目标寄存器字段。其 `out_expr` 仅标识所验证的表达式。汇编中的目标寄存器由整个方向的隐式 SSA 编号规则确定，不能改变二进制执行顺序。`SPLIT` 的分支局部定义先占用编号，随后为汇合输出分配编号；编号细节与不可达代码的处理见机器模型章。所有生成的事实均具有当前 scope。

下文以 $\mathrm{expr}(r)$ 和 $\mathrm{kind}(r)$ 表示事实引用的表达式与类型。表达式相等是精确有理数意义下逐项相等：常数项、变量编号和对应系数必须相同。它不是 ExprId 相同，也不允许把正比例表达式视为同一命题。

3.1.3　校验与提交

每条指令必须先完成载荷、引用、类型与数学前置条件检查，再提交可观察的状态变化。失败不能提交部分事实、部分映射或部分义务。`E_REF` 表示引用不存在、已死亡、来自未来或不可见；`E_KIND` 表示事实类型不符合该指令；`E_PROOF` 表示所声明的推导不成立。错误符号的完整定义见异常章。

执行只计算精确有理数。任何实现都不得用浮点近似决定表达式相等、符号、整数性、取整或分支覆盖。宿主整数溢出必须提升到精确表示或报告资源错误，不能产生环绕后的证明结果。

3.1.4 操作码总表

操作码	RPA 助记符	功能
0x00	nop	无状态变化
0x01	ret	完成一个执行方向
0x02	drop	释放当前 scope 的派生事实
0x10	lin.le	线性组合得到 LE
0x11	lin.eq	线性组合得到 EQ
0x12	eq.pair	由相反方向不等式得到 EQ
0x13	box.le	由声明变量界得到 LE
0x14	weaken.le	放松一个 LE
0x15	close.false	以常数矛盾关闭当前域
0x20	int.aff	组合整数事实得到 INT
0x21	round.hi	整数表达式上界取整
0x22	round.lo	整数表达式下界取整
0x30	map.copy	定义输出变量的复制映射
0x31	map.const	定义输出变量的常数映射
0x32	map.aff	定义输出变量的仿射映射
0x40	check.row.hi	解除输出行上界义务
0x41	check.row.lo	解除输出行下界义务
0x42	check.row.eq	解除输出等式行义务
0x43	check.var.hi	解除输出变量上界义务
0x44	check.var.lo	解除输出变量下界义务
0x45	check.var.int	解除输出变量整数性义务
0x46	check.obj.eq	解除目标相等义务
0x47	check.obj.le	解除目标不增义务
0x50	split.bin	对显式 binary 输入变量分类
0x51	split.int.thr	对整数表达式按阈值分类
0x52	split.int.range	枚举整数表达式的有限区间
0x53	split.sign	对实仿射表达式按符号分类
0x54	yield	提交一个可达分支的汇合事实

0x80--0xBF 为 batch/fused 扩展保留，0xF0 为 theorem 调用保留。这些编码不属于本章的核心指令集。未声明、未支持或未知操作码必须拒绝，不能当作 nop 跳过。

3.2 一般指令

3.2.1 NOP：无操作

```
nop
```

编码为 opcode=0x00、flags=0、aux=0、payload_bytes=0。NOP 不读取事实，也不生成事实、映射或义务；执行继续到下一条指令。非零保留字段或载荷以 E_FORMAT 拒绝。NOP 不能替代 root 或分支所要求的终结指令。

3.2.2 RET：完成当前方向

```
ret
```

编码为 opcode=0x01、flags=0、aux=0、payload_bytes=0。RET 必须位于当前方向 root tape 的最后一条指令位置，且 scope stack 必须只包含 root。

对仍可达的 root，RET 检查：所有输出变量都有完整定义；所有要求的输出行、变量界及整数性义务均为 DONE；contract 要求的目标关系已完成。完整定义包括可用的单一表达式映射，以及经所有可达分支封存、且不再有依赖义务的分段映射。不得把封存状态误判为未映射状态。

成功后该方向成为 VERIFIED。缺失映射以 E_MAP 拒绝，未完成义务以 E_OBLIGATION 拒绝，错误 scope 或非终结位置以 E_SCOPE 或 E_FORMAT 拒绝。RET 不接受分支事实返回，也不能结束嵌套分支。root 已被证明不可达时采用机器模型章规定的空可行域结束规则。

3.2.3 DROP：释放事实区间

```
drop %100 .. %499
```

编码为 opcode=0x02、flags=0、aux=0，载荷为 8 字节：

```
first: u32;
count: u32;
```

语义作用于半开区间 $[\text{first}, \text{first} + \text{count})$ 内的派生 FactId。count 必须大于零。整个区间必须无溢出、全部已定义、全部活跃，且全部属于当前 scope。检查全部通过后一次性标记为 DEAD，随后任何事实引用都不能重新激活这些编号。DROP 不释放输入事实或分支假设。

修订提案 P-03。为避免一个分支删除兄弟分支仍需读取的祖先事实，v0.1 草案限制 DROP 只能释放当前 scope 所拥有的派生事实。该限制是待确认的 scope 管理规则。

空区间或算术越界以 E_FORMAT 拒绝；未定义或已死亡的事实以 E_REF 拒绝；释放祖先或其他 scope 的事实以 E_SCOPE 拒绝。DROP 改变逻辑活跃状态，但实现能否回收物理槽位取决于其事实存储结构，不能仅由该指令推出峰值内存界。

3.3 线性事实指令

3.3.1 LIN.LE：非负不等式组合

```
%7 = lin.le $e31 {
    +1   @in.r[17].hi
    +2   %3
    -3/5 @in.r[91].eq
}
```

编码为 opcode=0x10、flags=0、aux=n。载荷长度必须为 $4 + 8n$ 字节：

```
out_expr: u32;
terms[n]: { fact: FactRef; multiplier: ScalarRef; };
```

每个输入必须为 LE 或 EQ。LE 输入的系数必须非负，EQ 输入的系数可为任意有理数。所有引用即使系数为零也必须有效，所有 scalar 也必须具有合法 canonical 编码。令输入表达式为 e_i、系数为 a_i，kernel 检查

$$e_{\text{out}} = \sum_{i=0}^{n-1} a_i e_i.$$

成功生成 LE(e_{out})。$n = 0$ 合法，但只可生成零表达式的 LE。重复输入引用按其实际出现次数参与组合，不改变推理规则。

输入 INT 以 `E_KIND` 拒绝；LE 的负系数、组合结果不相等以 `E_PROOF` 拒绝；非法引用以 `E_REF` 拒绝，非法 scalar 以 `E_SCALAR` 拒绝。

3.3.2 LIN.EQ：等式线性组合

```
%8 = lin.eq $e42 {
    +1 @in.r[12].eq
    -3 @in.r[19].eq
}
```

编码为 `opcode=0x11`、`flags=0`、`aux=n`；载荷字段与 LIN.LE 相同，长度为 $4 + 8n$ 字节。所有输入必须为 EQ，所有系数可为任意精确有理数。检查

$$e_{\text{out}} = \sum_i a_i \operatorname{expr}(r_i)$$

后生成 EQ(e_{out})。空组合仅能产生 EQ(0)。非 EQ 输入以 `E_KIND` 拒绝，结果不相等以 `E_PROOF` 拒绝。

3.3.3 EQ.PAIR：相反不等式构成等式

```
%9 = eq.pair $e17, %3, %8
```

编码为 `opcode=0x12`、`flags=0`、`aux=0`，载荷为 12 字节：

```
out_expr:      u32;
positive_fact: FactRef;
negative_fact: FactRef;
```

两个输入必须为 LE，并且分别满足

$$\operatorname{expr}(r_+) = e_{\text{out}}, \qquad \operatorname{expr}(r_-) = -e_{\text{out}}.$$

成功生成 EQ(e_{out})。不接受只差非单位比例的表达式；这种情况必须先由 LIN.LE 显式调整。类型错误以 `E_KIND` 拒绝，任一精确比较失败以 `E_PROOF` 拒绝。

3.3.4 BOX.LE：声明变量界检查

```
%10 = box.le $e73
```

编码为 `opcode=0x13`、`flags=0`、`aux=0`，载荷仅含 `out_expr:u32`，长度为 4 字节。对

$$e = c + \sum_i a_i x_i$$

使用当前输入模型声明的界 l_i, u_i，计算

$$U = c + \sum_{a_i>0} a_i u_i + \sum_{a_i<0} a_i l_i.$$

若每个实际使用的界均有限且 $U \le 0$，则生成 LE(e)。canonical 表达式不含零系数，因此不存在以 $0 \cdot \infty$ 绕过检查的情况。常数表达式直接检查 $c \le 0$。

本指令不读取派生变量界，也不使用 branch assumption 自动收紧界。需要这些事实时应使用 LIN.LE。所需界不存在、无限或计算结果为正时以 `E_PROOF` 拒绝；非法变量编号或表达式编号以 `E_REF` 拒绝。

3.3.5 WEAKEN.LE：常数放松

```
%11 = weaken.le $e91, %7, 3/2
```

编码为 `opcode=0x14`、`flags=0`、`aux=0`，载荷为 12 字节：

```
out_expr: u32;
fact:     FactRef;
delta:    ScalarRef;
```

输入必须为 LE，且 $\delta \ge 0$。kernel 检查

$$e_{\text{out}} = \text{expr}(r) - \delta$$

后生成 LE(e_{out})。$\delta = 0$ 合法。非 LE 输入以 `E_KIND` 拒绝；负 δ 或表达式不匹配以 `E_PROOF` 拒绝。

3.3.6 CLOSE.FALSE：关闭矛盾域

```
close.false %27
```

编码为 `opcode=0x15`、`flags=0`、`aux=0`，载荷仅含 `fact:FactRef`，长度为 4 字节。输入必须是无变量项的常数事实，且满足以下任一条件：

$$\text{LE}(c) \text{ 且} c > 0, \qquad \text{EQ}(c) \text{ 且} c \neq 0.$$

成功标记当前分支或 root 为 UNREACHABLE，不产生事实，不再要求当前域完成映射、义务或 YIELD。它必须是该显式指令块的最后一条指令。

非 LE/EQ 类型以 `E_KIND` 拒绝；非常数表达式、LE($c \le 0$) 或 EQ(0) 以 `E_PROOF` 拒绝。INT(1/2) 不是本指令接受的矛盾形式。root 关闭表示输入可行域为空，该方向的全称映射义务因而成立；另一个方向仍须独立验证。

3.4 整数事实指令

3.4.1 INT.AFF：整数仿射组合

```
%12 = int.aff $e104, 7 {
    +3 %3
    -2 %7
}
```

编码为 opcode=0x20、flags=0、aux=n，载荷长度为 $8+8n$ 字节：

```
out_expr:       u32;
integer_offset: ScalarRef;
terms[n]: { fact: FactRef; multiplier: ScalarRef; };
```

offset 必须为整数。INT 输入的 multiplier 必须为整数；EQ 输入的 multiplier 可为任意有理数。LE 输入不允许。kernel 验证

$$e_{\text{out}} = k + \sum_{\text{kind}(r_i)=\text{INT}} n_i \,\text{expr}(r_i) + \sum_{\text{kind}(r_j)=\text{EQ}} \mu_j \,\text{expr}(r_j),$$

其中 $k, n_i \in \mathbb{Z}$、$\mu_j \in \mathbb{Q}$。成功生成 INT(e_{out})。空组合可生成任意整数常数的 INT。

修订提案 P-04。原始草案只允许 INT 输入。本手册提出允许 EQ 项参与 INT.AFF：EQ 项恒等于零，可在不引入新操作码的情况下把整数性沿已证明等式传递，例如由 INT(z) 与 EQ(x-z) 推出 INT(x)。

LE 输入以 E_KIND 拒绝；非整数 offset、INT 项的非整数系数或结果不相等以 E_PROOF 拒绝。输入表达式常数项本身可以是有理数，只要输入 INT 事实已经成立；kernel 不得把“所有表达式系数必须是整数”作为此指令的额外限制。

3.4.2 ROUND.HI：整数上界取整

```
%13 = round.hi $e_x_minus_7, %4, %7
```

编码为 opcode=0x21、flags=0、aux=0，载荷为 12 字节：

```
out_expr:  u32;
bound_fact: FactRef;
int_fact:   FactRef;
```

令 $g = \text{expr}(r_{\text{int}})$、$b = \text{expr}(r_{\text{bound}})$。输入类型必须分别为 INT 和 LE，并且 $g - b$ 必须是常数 β。kernel 检查

$$b = g - \beta, \qquad e_{\text{out}} = g - \lfloor \beta \rfloor.$$

成功生成 LE(e_{out})。g 可为任意已证明整数的仿射表达式，不局限于单变量；其常数项也参与相减。取整是数学 floor，对负数同样向负无穷方向取整。

类型不匹配以 E_KIND 拒绝；$g - b$ 含变量项或输出不匹配以 E_PROOF 拒绝。例：$g = x$、$b = x - 37/5$ 得到 $x - 7 \le 0$。

3.4.3 ROUND.LO：整数下界取整

```
%14 = round.lo $e_neg_x_plus_8, %5, %7
```

编码为 opcode=0x22、flags=0、aux=0；载荷字段与 ROUND.HI 相同，长度为 12 字节。输入 r_{bound} 必须为 LE，r_{int} 必须为 INT。令它们的表达式为 b, g，则 $b + g$ 必须是常数 β，且

$$b = -g + \beta, \qquad e_{\text{out}} = -g + \lceil \beta \rceil.$$

成功生成 LE(e_{out})。数学 ceil 对负数向正无穷方向取整。例：$-x+37/5 \le 0$ 与 INT(x) 得到 $-x+8 \le 0$。类型不匹配以 `E_KIND` 拒绝；和不为常数或输出不匹配以 `E_PROOF` 拒绝。

3.5 映射指令

3.5.1 映射指令的公共条件

本节指令只定义输出变量的取值表达式，不自动生成 LE、EQ、INT，也不解除任何输出义务。载荷中的输入变量与表达式都属于当前方向的输入坐标系。

tape 中的映射指令只能赋值给 map section 中声明为 DEFER、且在当前 scope 状态中为 UNASSIGNED 的输出变量。父 scope 已有 KNOWN 映射的变量不能被子分支重定义。各兄弟分支从同一父状态开始，因此可以分别赋值给同一个尚未定义的 DEFER 变量。重复赋值、非 DEFER 赋值或重写封存的分段映射均以 `E_MAP` 拒绝。

分支映射在汇合时的相等性、封存与义务依赖规则见机器模型章；任一 CHECK 需要 KNOWN 映射才能实际完成代换。

3.5.2 MAP.COPY：复制输入变量

```
map.copy @out.v[17], @in.v[9]
```

编码为 `opcode=0x30`、`flags=0`、`aux=0`，载荷为 8 字节：

```
out_var: u32;
in_var:  u32;
```

成功将 $E_{17}(x) = x_9$ 存为 KNOWN。两个索引必须分别落在输出、输入变量范围内，否则以 `E_REF` 拒绝。输入变量的 domain 不自动转移到输出变量；输出 bounds 和 integrality 仍必须分别检查。

3.5.3 MAP.CONST：常数映射

```
map.const @out.v[18], 0
```

编码为 `opcode=0x31`、`flags=0`、`aux=0`，载荷为 8 字节：

```
out_var: u32;
value:   ScalarRef;
```

成功将 $E_{18}(x) = \text{value}$ 存为 KNOWN。value 可以是任意合法有理数；是否符合输出变量 bounds 和 domain 由 CHECK 决定。输出索引越界以 `E_REF` 拒绝，非法 scalar 编码以 `E_SCALAR` 拒绝。

3.5.4 MAP.AFF：仿射映射

```
map.aff @out.v[19], $e37
```

编码为 `opcode=0x32`、`flags=0`、`aux=0`，载荷为 8 字节：

```
out_var: u32;
expr:    u32;
```

成功将输出变量的 KNOWN 映射设为 $E_{19}(x) = e_{37}(x)$。输出索引或表达式编号无效以 `E_REF` 拒绝。表达式不得引用输出变量，不能通过 MAP.AFF 构造递归映射。

3.6 模型义务检查指令

3.6.1 CHECK 的公共语义与 EXACT 模式

CHECK 在执行时由 kernel 从输出模型和当前映射构造待证表达式 $P(x)$。它不接受证书自行声明目标表达式。参与代换的每个输出变量都必须为 KNOWN；缺失或 SEALED_PIECEWISE 映射以 `E_MAP` 拒绝。已封存映射对应的义务必须已经完成，不得用于新的代换。

所有 CHECK 的 `aux=0`。普通模式 `flags=0`，模型对象检查载荷为

```
out_index: u32;
fact:      FactRef;
```

共 8 字节；目标检查仅有 `fact:FactRef`，共 4 字节。普通模式要求事实类型与该指令一致，且 $\mathrm{expr}(r) = P$。成功将对应 PENDING 义务置为 DONE，不产生可引用事实。对已经 DONE 的义务再次执行 CHECK 必须以 `E_OBLIGATION` 拒绝；分支继承的 DONE 义务同样不能重复检查。

EXACT 模式仅置 `flags` 的 bit 0，其他位必须为零。模型对象检查载荷仅有 `out_index:u32`，共 4 字节；目标检查没有载荷。EXACT 完全由 kernel 执行以下确定性检查：

- EQ 检查要求 P 为零表达式。
- LE 检查要求 P 不含变量项且其常数 $c \le 0$。
- INT 检查要求 P 的常数项为整数，并且每个非零系数为整数、对应输入变量在模型中显式声明为 integer 或 binary。

修订提案 P-05。原始草案使用 exact 汇编 pseudo-op，但未指定二进制编码。本手册提出 CHECK 的 bit 0 作为 EXACT 模式。该模式不信任 assembler 的判断，也不生成隐式 FactId；所有表达式由 kernel 重新代换并检查。

EXACT 不进行 BOX.LE、等式消元、推理搜索或利用派生整数事实。CHECK 对一个不存在的模型侧或不属于当前 contract 的义务以 `E_OBLIGATION` 或 `E_CONTRACT` 拒绝；事实类型错误以 `E_KIND` 拒绝；表达式比较或 EXACT 检查失败以 `E_PROOF` 拒绝。成功标记具有当前分支的可行域条件，在汇合时对所有可达分支取交集。

3.6.2 CHECK.ROW.HI：输出行上界

```
check.row.hi @out.r[17], %81
check.row.hi @out.r[17], exact
```

操作码为 `0x40`，采用本节公共 CHECK 编码。输出行 r 必须有有限上界 u_r，且该行侧必须存在于义务集合。令行系数为 A_{rj}，构造

$$P = \sum_j A_{rj} E_j(x) - u_r.$$

普通模式要求 LE(P)，EXACT 模式要求 P 是非正的常数。成功解除该行的 upper-side 义务。不存在的上界、不被要求的侧或越界行号必须拒绝。规范化等式行使用 CHECK.ROW.EQ，不把等式义务隐式拆成两个独立侧。

3.6.3 CHECK.ROW.LO：输出行下界

```
check.row.lo @out.r[17], %82
check.row.lo @out.r[17], exact
```

操作码为 0x41，采用公共 CHECK 编码。输出行必须有有限下界 l_r，构造

$$P = -\sum_j A_{rj} E_j(x) + l_r.$$

普通模式要求 LE(P)，EXACT 模式检查常数非正。成功解除 lower-side 义务。下界不存在、行号越界或非此侧义务按公共拒绝规则处理。行常数若存在于 canonical 模型中，必须在正规化行数据里精确计入，不得重复加减。

3.6.4 CHECK.ROW.EQ：输出等式行

```
check.row.eq @out.r[17], %91
check.row.eq @out.r[17], exact
```

操作码为 0x42，采用公共 CHECK 编码。输出行必须为显式 canonical equality $\sum_j A_{rj} y_j = b_r$。构造

$$P = \sum_j A_{rj} E_j(x) - b_r.$$

普通模式要求 EQ(P)，EXACT 模式要求 $P = 0$。成功解除该行的 equality 义务。即使两个 LE 事实分别证明了两侧，也必须先使用 EQ.PAIR 得到 EQ 后才能作为普通模式输入。

3.6.5 CHECK.VAR.HI：输出变量上界

```
check.var.hi @out.v[17], %22
check.var.hi @out.v[17], exact
```

操作码为 0x43，采用公共 CHECK 编码。输出变量 j 必须有有限上界 u_j，构造 $P = E_j(x) - u_j$。普通模式要求 LE(P)，EXACT 模式要求非正的常数。成功解除变量 upper-bound 义务。binary 变量的 canonical bounds 也通过该指令或普通证明检查，不由 MAP 自动解除。

3.6.6 CHECK.VAR.LO：输出变量下界

```
check.var.lo @out.v[17], %21
check.var.lo @out.v[17], exact
```

操作码为 0x44，采用公共 CHECK 编码。输出变量 j 必须有有限下界 l_j，构造 $P = -E_j(x) + l_j$。普通模式要求 LE(P)，EXACT 模式要求非正的常数。成功解除变量 lower-bound 义务。

3.6.7 CHECK.VAR.INT：输出变量整数性

```
check.var.int @out.v[17], %23
check.var.int @out.v[17], exact
```

操作码为 0x45，采用公共 CHECK 编码。输出变量必须显式声明为 integer 或 binary，构造 $P = E_j(x)$。普通模式要求 INT(P)，EXACT 模式执行本节规定的声明整数变量线性组合检查。成功解除该变量的 integrality 义务。

EXACT 能接受 $3x_0 - 2x_1 + 7$（x_0, x_1 均显式整数），但不能仅凭已有 EQ 或 INT 事实接受一个连续变量 x_2。后者应先生成 INT 事实并使用普通模式。对连续输出变量调用本指令没有对应义务，必须拒绝。

3.6.8 CHECK.OBJ.EQ：目标相等

```
check.obj.eq %101
check.obj.eq exact
```

操作码为 0x46，普通载荷为 fact:FactRef（4 字节），EXACT 载荷为 0 字节，aux=0。对当前方向已经统一为最小化的目标，构造

$$D(x) = f_{\text{out}}(E(x)) - f_{\text{in}}(x).$$

两个目标的所有常数 offset 都必须包括在 D 内。普通模式要求 EQ(D)，EXACT 要求 $D = 0$。当当前方向的目标 relation 为 EQ 时，成功完成目标相等义务；当 relation 为 LE 时，它作为更强的证明完成目标不增义务。此规则不改变证书声明或调用者所要求的 contract。relation 为 NONE 时没有可解除的目标义务，本指令必须以 E_OBLIGATION 拒绝。

缺失必要映射以 E_MAP 拒绝，不存在或已完成的目标义务以 E_OBLIGATION 拒绝。证书所声明的 contract 与调用者请求不一致属于 E_CONTRACT，应在执行之前拒绝。遗漏常数 offset 的事实会因表达式不匹配而以 E_PROOF 拒绝。

3.6.9 CHECK.OBJ.LE：目标不增

```
check.obj.le %101
check.obj.le exact
```

操作码为 0x47，编码与 CHECK.OBJ.EQ 相同。构造相同的 $D = f_{\text{out}} \circ E - f_{\text{in}}$。普通模式要求 LE(D)，EXACT 要求 D 为非正的常数。当前方向必须要求 LE relation，成功完成目标不增义务；relation 为 NONE 或 EQ 时以 E_OBLIGATION 拒绝。

CHECK.OBJ.LE 不能解除目标 equality 要求，即使证书在某处还证明了相反不等式；应显式生成 EQ 并使用 CHECK.OBJ.EQ。最大化模型必须先按照 contract 的目标规范化规则转换方向，不能令同一个操作码在运行时隐式反转不等号。

3.7 结构化分类与汇合指令

3.7.1 SPLIT 公共格式

SPLIT 没有跳转目标。其载荷内包含完整、有界、顺序排列的子指令块；kernel 依次检查每个块，兄弟块共享同一父状态快照。任何指令都不能跨出所属 byte range。

`flags=0`，`aux=q` 是每个可达分支必须提交的 YIELD 数量，同时也是汇合后产生的事实数量。允许 $q = 0$：此时分支只用于映射和义务检查。每个 `branch_bytes` 必须大于零且为 8 的倍数，包括该块内每条指令自己的对齐填充。每个子块必须以 YIELD 或 CLOSE.FALSE 结束，RET 不可出现在子块内。终结指令之后不得在同一子块内放置额外指令。

分支假设通过 `FactRef` tag 7 暴露，`@case[0]` 指当前分支假设；更大的 depth offset 指可见祖先分类假设。它们不是派生 FactId，不需要 LOAD，也不能被 DROP。每个 split 进入时都建立独立子 scope。

3.7.2 SPLIT 公共汇合语义

每个可达分支的 YIELD 必须具有相同 arity q，并且每个位置的 FactKind 与精确表达式相同。所有分支验证完成后，在父 scope 中为每个 yield 位置生成一个新事实。

- 没有可达分支：父域成为 UNREACHABLE，不提交 q 个可用事实。
- 只有一个可达分支：它的 q 个命题成为父 scope 的汇合事实。
- 至少两个可达分支：逐位置比较命题后生成 q 个父事实。

某个分支 CLOSE.FALSE 时不需要 YIELD，也不参与义务交集或映射比较。所有分支局部派生事实在汇合后失去可见性。已完成义务按可达分支取交集；不能把不同分支分别完成的两个义务合并为全局完成。

映射只在可达分支中比较。若一个变量在部分可达分支已定义、在另一些可达分支未定义，必须立即以 `E_MAP` 拒绝；所有可达分支都未定义时保留 UNASSIGNED。相同的已定义映射可保留为 KNOWN；不同映射只有在每支都定义完整、且所有依赖它的义务均在每支完成时才能封存为 SEALED_PIECEWISE，否则以 `E_MAP` 或 `E_OBLIGATION` 拒绝。封存允许 RET，但禁止后续代换。

若嵌套 split 已证明整个当前域不可达，后续字节只接受结构、编码及索引格式检查，不进行依赖于活跃事实的语义执行，也不制造可引用事实。仍须验证该块具有合法结构终结；不可达状态不能成为跳过未知操作码或越界数据的手段。隐式 FactId 编号不因该语义跳过而改变。

3.7.3 SPLIT.BIN：显式二元变量分类

```
(%31) = split.bin @in.v[7] {
case 0:
    %27 = lin.eq $zero { }
    yield %27
case 1:
    %30 = lin.eq $zero { }
    yield %30
}
```

本例的表达式别名 `zero` 表示零表达式，因此两支均返回 EQ(0)。一般情况下，两个分支的 YIELD 必须证明相同命题，不能仅因引用写法相同而通过汇合。

操作码为 `0x50`、`aux=q`，载荷固定头 16 字节：

```
var_id:        u32;
branch0_bytes: u32;
branch1_bytes: u32;
reserved:      u32;
branch0[branch0_bytes];
branch1[branch1_bytes];
```

payload_bytes 必须等于 $16 + B_0 + B_1$。输入变量必须显式声明为 binary，不能只凭派生 INT 与界推断其 binary domain。case 0 假设为 EQ(x_v)，case 1 为 EQ($x_v - 1$)。这两支覆盖输入声明的二元域。

变量越界以 E_REF 拒绝，非 binary domain 以 E_KIND 拒绝，子块长度或保留字段错误以 E_FORMAT 拒绝。汇合类型、表达式、arity 不一致以 E_BRANCH 拒绝。输入模型对 binary 的 canonical domain 定义必须包含取值限于 $\{0, 1\}$ 的语义，不能仅把 binary 作为无约束整数标签。

3.7.4 SPLIT.INT.THR：整数阈值分类

```
(%40) = split.int.thr %7, 13 {
case le:
    yield %38
case ge:
    yield %39
}
```

操作码为 0x51、aux=q，载荷固定头 16 字节：

```
int_fact:      FactRef;
threshold:     ScalarRef;
branch0_bytes: u32;
branch1_bytes: u32;
branch0[branch0_bytes];
branch1[branch1_bytes];
```

payload_bytes 为 $16 + B_0 + B_1$。输入必须为 INT(g)，threshold 必须为整数 k。第一支 case le 假设为 LE($g - k$)，第二支 case ge 假设为 LE($-g + k + 1$)。它们分别代表 $g \le k$ 和 $g \ge k + 1$，无需上下界即可覆盖全部整数。

非 INT 引用以 E_KIND 拒绝，非整数 threshold 以 E_PROOF 拒绝。$k + 1$ 必须精确计算，即使 k 无法放入宿主固定宽度整数。其他结构与汇合错误按 SPLIT 公共规则处理。

3.7.5 SPLIT.INT.RANGE：整数有限区间枚举

```
(%50) = split.int.range %int_x, %lo_x, %hi_x, 3, 7 {
case 3: yield %43
case 4: yield %44
case 5: yield %45
case 6: yield %46
case 7: yield %47
}
```

操作码为 0x52、aux=q，载荷固定头 24 字节：

```
int_fact:    FactRef;
lo_fact:     FactRef;
hi_fact:     FactRef;
lower:       ScalarRef;
upper:       ScalarRef;
branch_count: u32;
branch_bytes[branch_count]: u32;
table_padding: zero bytes to 8-byte alignment;
branches: concatenated complete instruction blocks;
```

令 N = branch_count，表区结束地址向上对齐至 8 字节；因固定头为 24 字节，表填充长度为 $(8-(24+4N) \bmod 8) \bmod 8$。载荷总长度必须为

$$\mathrm{align}_8(24+4N)+\sum_{i=0}^{N-1} B_i.$$

upper 和 lower 必须为整数 U, L，并且 $L \le U$、$N = U - L + 1$。N 必须能以 u32 表示，并满足文件和执行资源限制。

输入必须精确具有以下类型与表达式：

$$\mathrm{INT}(g), \qquad \mathrm{LE}(-g+L), \qquad \mathrm{LE}(g-U).$$

不接受更宽松或缩放后的界，assembler 可以先通过核心指令构造要求的精确事实。第 i 个子块对应 $k = L+i$，分支假设为 $\mathrm{EQ}(g-k)$。没有可省略的 case，也没有默认分支。允许 $L = U$，此时只有一个子块。

类型错误以 E_KIND 拒绝；非整数端点、逆序区间、界表达式不匹配以 E_PROOF 拒绝；branch count、表 padding、累加长度或块边界不一致以 E_FORMAT 拒绝。资源不足不能降级为少枚举若干分支。

3.7.6 SPLIT.SIGN：实表达式符号分类

```
() = split.sign $e {
case le:
    map.aff @out.v[7], $neg_e
    yield
case ge:
    map.aff @out.v[7], $e
    yield
}
```

本例只展示不同分支映射的语法；所有依赖输出变量 7 的义务必须在各支 YIELD 之前完成。

操作码为 0x53、aux=q，载荷固定头 16 字节：

```
expr:          u32;
branch0_bytes: u32;
branch1_bytes: u32;
reserved:      u32;
branch0[branch0_bytes];
branch1[branch1_bytes];
```

payload_bytes 为 $16 + B_0 + B_1$。令表达式为 g，case le 假设为 LE(g)，case ge 假设为 LE($-g$)。两支覆盖全部实数；$g = 0$ 同时属于两支，允许分段映射在该交集上给出不同但分别合法的输出解。

不要求 g 为整数或预先得到其他事实。表达式编号越界以 E_REF 拒绝；保留字段、子块格式错误按公共格式规则拒绝。两支重叠不允许以单支验证替代另一支。

3.7.7 YIELD：分支事实返回

```
yield %17, %19
yield
```

编码为 opcode=0x54、flags=0、aux=n，载荷为 $4n$ 字节：

```
refs[n]: FactRef;
```

YIELD 必须是一个子块的最后一条指令，不允许出现在 root。n 必须等于所属 SPLIT 的 q。每个引用必须有效、活跃且在当前分支可见；允许返回祖先事实、输入事实或当前 case 事实，只要最终汇合满足命题一致条件。静态输入事实的表达式可直接由不可变输入模型表示，汇合不要求在表达式池中查找或插入一个相同 ExprId。

YIELD 不在子 scope 中生成新的事实；它记录有序命题列表供父 split 比较，并结束该分支。空 YIELD 用于 q=0。父 split 完成之前，这些命题不能被另一分支引用。关闭为 UNREACHABLE 的分支不需要执行 YIELD。

非分支位置或开放 scope 错误以 E_SCOPE 拒绝；不位于块末尾或载荷长度错误以 E_FORMAT 拒绝；arity 不匹配以 E_BRANCH 拒绝；非法、dead 或超出 scope 的引用以 E_REF 拒绝。

3.8 逐指令载荷尺寸速查

下表中的尺寸不包括 8 字节指令头和指令尾部的对齐填充。n 表示 aux，B_i 表示完整子指令块字节数。所有尺寸公式均必须经过无溢出的边界检查。

指令	aux	payload_bytes
NOP、RET	0	0
DROP	0	8
LIN.LE、LIN.EQ	term count	$4 + 8n$
EQ.PAIR	0	12
BOX.LE	0	4

指令	aux	payload_bytes
WEAKEN.LE	0	12
CLOSE.FALSE	0	4
INT.AFF	term count	$8 + 8n$
ROUND.HI、ROUND.LO	0	12
MAP.COPY、MAP.CONST、MAP.AFF	0	8
CHECK.ROW / CHECK.VAR，普通	0	8
CHECK.ROW / CHECK.VAR，EXACT	0	4
CHECK.OBJ，普通	0	4
CHECK.OBJ，EXACT	0	0
SPLIT.BIN、SPLIT.INT.THR、SPLIT.SIGN	yield arity	$16 + B_0 + B_1$
SPLIT.INT.RANGE	yield arity	$\mathrm{align}_8(24 + 4N) + \sum_i B_i$
YIELD	reference count	$4n$

每个 opcode 的 arity、类型与命题检查仍以对应参考条目为准。载荷尺寸正确只是进入执行语义检查的必要条件，并不构成证明成功。

第四章　RPX 二进制文件格式

4.1　通用编码约定

所有多字节整数采用 little endian。字段宽度是文件协议的一部分，与宿主 C ABI、Rust 布局、指针宽度无关。实现必须按偏移解码字段；不得把宿主结构体的隐式 padding 当成文件字节。`u8/u16/u32/u64` 分别表示 8、16、32、64 位无符号整数。没有浮点字段、指针、原生对象或可执行宿主机器码。

修订提案 P-07。本章固定原草案未明确的 header padding、ScalarPool 记录、map 记录及各 split 的长度布局。这些字节布局共同构成 D0 的候选 wire format；不得在实现中分别选择不同解释。

设 $\mathrm{align}_a(z)$ 为大于等于 z 的最小 a 的倍数。所有加法、乘法和向上对齐都必须检测溢出，且转换为宿主索引前必须验证可表示性。不能先使用 wrapping arithmetic 再比较文件边界。所有明确规定的 padding 与 reserved 字节必须为零；未知的逻辑 flag 必须拒绝。

文件由 128 B header、section directory 和若干 section 构成。directory 从字节 128 开始。各 section 的起始偏移都是 64 的倍数。section 之间的未使用间隙必须为零。header 的 `file_bytes` 必须恰好等于实际读取的文件长度；不允许尾随未声明数据。

4.2　Header

固定 header 长度为 128 B，`header_units=16`。

偏移	字节数	字段	v0.1 要求
0	4	`magic`	字节 `52 50 58 00`，即 RPX 后接 NUL。
4	1	`endian`	1；只接受 little endian。
5	1	`major`	0。
6	1	`minor`	1。
7	1	`header_units`	16，以 8 B 为单位。
8	4	`flags`	0；全部保留。
12	4	`reserved0`	0；显式字段，不是 ABI padding。
16	8	`file_bytes`	实际文件长度。
24	32	`source_sha256`	canonical source model 的 SHA-256 原始 digest。
56	32	`target_sha256`	canonical target model 的 SHA-256 原始 digest。
88	4	`contract_flags`	见下文的 contract 编码。
92	4	`section_count`	directory entry 数量。
96	8	`section_dir_offset`	128。
104	24	`reserved1`	全零。

SHA 字段不是 UTF-8 十六进制字符串。assembler 将文本中的 64 个 hexadecimal 字符转换为 32 个 digest 字节。source 与 target 的顺序永远不因执行方向交换；交换的是 VM 的 input/output model 绑定。原草案 C 结构体的字段若直接逐项写出只有 124 B。本布局用 `reserved0` 显式补齐 4 B，并固定偏移。

4.2.1 Contract word

位	含义
0	启用 Forward。
1	启用 Reverse。
2	`opt-equivalent` profile。
3	`feasibility-equivalent` profile。
5:4	Forward objective relation：0 NONE，1 EQ，2 LE，3 保留。
7:6	Reverse objective relation：0 NONE，1 EQ，2 LE，3 保留。
31:8	保留，必须为零。

本草案的两个 profile 均要求启用两个方向。位 2 和位 3 必须恰有一个为 1。`opt-equivalent` 要求每个方向的 objective relation 均为 EQ 或 LE。`feasibility-equivalent` 允许 NONE、EQ 或 LE；如果选择 EQ/LE，相关 obligation 仍必须检查。位模式不符合这些条件时为 `E_CONTRACT`。

例：双向 objective equality 的 optimum-equivalence 编码为 `0x00000057`，文件字节为 `57 00 00 00`。header 声明必须与调用者要求相容，不能用更弱的 profile 绕过调用者请求。将来若加入仅单方向的 implication profile，必须另行定义版本或明确扩展，不能悄悄放宽本段约束。

4.3 Section directory

每个 entry 长度为 32 B。

entry 内偏移	宽度	字段
0	2	`type:u16`
2	2	`flags:u16`
4	4	`alignment:u32`
8	8	`offset:u64`
16	8	`length:u64`
24	8	`count:u64`

directory 长度为 32 × `section_count`。其范围必须包含于文件。`alignment` 必须为 64，`offset` 必须为 64 的倍数。`offset+length` 必须无溢出且不超过 `file_bytes`。任何非空 section 不得与 header、directory 或其他 section 相交。entry 按 `offset` 非降序排列；同一 type 不得重复。零长度 section 可以共享一个对齐偏移，因为其字节范围为空。所有 section 的 offset 必须不小于 directory 末尾向上对齐至 64 B 后的偏移，空 section 亦然。directory 和最后一个 section 之间的 padding 也属于必须为零的未使用间隙。

`flags` 的位 0 表示 OPTIONAL，其他位保留。所有已知的逻辑 section 必须 `flags=0`；WITNESS 与 DEBUG 必须 `flags=1`。未知 type 只有在 OPTIONAL 为 1 时可跳过；跳过的数据绝不能参与证明。未知 mandatory section 为 `E_UNSUPPORTED`。未知 opcode 始终拒绝，不能由 OPTIONAL section flag 覆盖。

type	名称	count 的单位	存在性与布局
`0x0001`	SCALAR_POOL	scalar entry	必需；可空，变长记录。
`0x0010`	FWD_EXPR_INDEX	expression	Forward 必需；16 B/entry。

type	名称	count 的单位	存在性与布局
0x0011	FWD_EXPR_TERMS	term	Forward 必需；8 B/entry。
0x0020	REV_EXPR_INDEX	expression	Reverse 必需；16 B/entry。
0x0021	REV_EXPR_TERMS	term	Reverse 必需；8 B/entry。
0x0030	FWD_MAP	map record	Forward 必需；记录流。
0x0031	REV_MAP	map record	Reverse 必需；记录流。
0x0040	FWD_TAPE	instruction	Forward 必需；包括所有嵌套指令。
0x0041	REV_TAPE	instruction	Reverse 必需；包括所有嵌套指令。
0x0050	WITNESS	byte	可选；count 必须等于 length。
0x00F0	DEBUG	byte	可选；count 必须等于 length。

两个方向的 pool 使用同一个 ScalarPool，却有各自独立的 ExprId 空间。ExprId 不得跨方向引用。TAPE 的 count 统计每个 split header 一次、每个 child instruction 一次，包括 unreachable tail 的指令；不统计 padding。基础 profile 启用双方向，因而共有 9 个必需 section；它们可以按 offset 排列，不要求按 type 排列。当 count 为 9 时，directory 在 416 处结束，首个 section 最早在 448 处开始。

4.4 ScalarRef 与 ScalarPool

4.4.1 内联整数

`ScalarRef` 是一个 u32。bit 31 为 1 时，低 31 位为二补码有符号整数。令 $s = \texttt{raw} \,\&\, \texttt{0x7FFFFFFF}$，则

$$\mathrm{decode}(\texttt{raw}) = \begin{cases} s, & s < 2^{30}, \\ s - 2^{31}, & s \geq 2^{30}. \end{cases}$$

范围是 $[-2^{30}, 2^{30} - 1]$，不是完整 i32 范围。

值	ScalarRef	little-endian 字节
0	0x80000000	00 00 00 80
1	0x80000001	01 00 00 80
-1	0xFFFFFFFF	FF FF FF FF
-2^{30}	0xC0000000	00 00 00 C0
$2^{30} - 1$	0xBFFFFFFF	FF FF FF BF

所有可内联的整数必须使用该编码，不能放入 ScalarPool。bit 31 为 0 时，全部 31 位为 ScalarPool index；index 必须小于 pool count。ScalarPool 的 count 不得超过 2^{31}，因为更大的索引没有可表示的 ScalarRef。

4.4.2 Pool 记录

设 scalar count 为 N。section 前 $16N$ 字节是索引表。每个 entry 包含 `data_offset:u64`、`numerator_bytes:u32` 和 `denominator_bytes:u32`，偏移依次为 0、8、12。`data_offset` 相对于 SCALAR_POOL section 起始位置，指向记录的 sign 字节。索引表之后是按 entry 顺序紧密连接的数据记录，不含内部 padding：

```
sign:u8 | numerator_magnitude[numerator_bytes] |
          denominator_magnitude[denominator_bytes]
```

sign=0 表示正，sign=1 表示负，其他值非法。两个 magnitude 都是 unsigned little-endian，不含符号位；长度必须大于零，最高有效字节不得为零。分母必须为正，分子必须非零，$\gcd(|p|, q) = 1$。零必须内联，因此没有 pool 内的正零、负零或 `0/1`。整数的 denominator magnitude 是单字节 `01`，且仅在超出内联范围时允许。

第一条记录的 data offset 必须为 $16N$，后续记录偏移必须等于前一条记录结束位置。最后一条记录必须恰好在 section 末尾结束。$N = 0$ 时 length 必须为零。ScalarPool 允许重复的 canonical rational；pool index 不构成数学数值身份。若实现使用 intern 或 hash 快路径，其结果仍必须服从精确数值相等。

例：一个只包含 1/10 的 pool，count=1，length=19。

```
10 00 00 00 00 00 00 00  01 00 00 00 01 00 00 00
00 01 0A
```

前 16 B 是 offset=16、numerator bytes=1、denominator bytes=1；后 3 B 是 sign=0、numerator=1、denominator=10。

4.5 Expression pool

每个方向有一个 expression pool，由该方向的 index section 与 terms section 组成。一个 pool entry 由一个 ExprIndex 记录及其 `term_offset`/`term_count` 指向的 term range 组成。ExprId 是该方向 index section 的零基 entry 序号。

4.5.1 ExprIndex

每个 ExprIndex 为 16 B：

偏移	宽度	字段
0	8	`term_offset:u64`
8	4	`term_count:u32`
12	4	`constant:ScalarRef`

每个方向的 expression count 不得超过 2^{32}，因为 ExprId 编码为 u32。`term_offset` 是相对于对应 TERMS section 的**字节偏移**，必须是 8 的倍数。`term_offset+8*term_count` 必须位于 TERMS section 内。index section 的 length 必须等于 $16 \times$ `count`。

4.5.2 ExprTerm

每个 ExprTerm 为 8 B：`var_id:u32` 位于偏移 0，`coefficient:ScalarRef` 位于偏移 4。terms section 的 length 必须等于 $8 \times$ `count`。每个 expression 的 variable ID 必须严格升序、位于当前 input model 范围内，且每个 coefficient 解码后非零。constant 与 coefficient 均须解码为 canonical rational。零表达式表示为 term count=0、constant= 内联零。

不同 expression 可以共享合法 term range；不同 ExprId 可以表示同一个 canonical expression。这里的 canonical 要求是数学表达式的有序、合并、精确表示，不要求 pool 全局去重。因此 `ExprId(a)==ExprId(b)` 可以作为相同表达式的充分条件，反向推论不成立。重复 pool entry 不得使 `EQ.PAIR`、CHECK 或 split merge 错误拒绝数学上精确相同的表达式。

4.6 FactRef

`FactRef` 是 u32，tag 为 bits 31:28，index 为 bits 27:0。

tag	文本形式	指向的事实
`0x0`	`%n`	当前方向 derived fact；index 是不可复用的 FactId。
`0x1`	`@in.r[n].hi`	输入 row 的有限 upper side，LE。
`0x2`	`@in.r[n].lo`	输入 row 的有限 lower side，LE。
`0x3`	`@in.r[n].eq`	输入 equality row，EQ。
`0x4`	`@in.v[n].hi`	输入 variable 的有限 upper bound，LE。
`0x5`	`@in.v[n].lo`	输入 variable 的有限 lower bound，LE。
`0x6`	`@in.v[n].int`	输入 integer/binary variable 的整数性，INT。
`0x7`	`@case[n]`	当前 scope 向外数 n 层的 split assumption。
`0x8--0xF`	无	保留；基础 v0.1 拒绝。

不存在的有限 side、非 equality 的 eq 引用、连续变量的 int 引用均非法。`@case` 等于 `@case[0]`；index 1 指父 split 的 assumption，不包括没有 assumption 的 root。derived fact 的 live、scope 与 future 检查见第二章。模型的可引用 row/variable 数量以及全部 syntactic derived fact slot 数量均不得超过 2^{28}。

4.7 Map section

map section 是一串独立记录，记录起始地址以 8 B 对齐。header 为 8 B：

```
kind:u8 | flags:u8 | reserved:u16 | payload_bytes:u32
```

flags 与 reserved 必须为零。record count 包含所有 map header。下一记录位置计算如下，padding 为零：

```
next_pc = align8(pc + 8 + payload_bytes)
```

未知 kind 拒绝；没有可跳过的 map 扩展。

kind	名称	payload（字段均为 u32）
`0x00`	COPY_RANGE	out start, in start, count, reserved=0；16 B。
`0x01`	COPY_PAIRS	pair count, reserved=0, 然后 pair count 组 (out var, in var)；$8+8n$ B。
`0x02`	CONST	out var, scalar ref；8 B。
`0x03`	AFF	out var, expr id；8 B。
`0x04`	DEFER	out start, count；8 B，覆盖连续输出变量。

range count 与 pair count 必须大于零。所有 range 的末端计算必须检测溢出，输入和输出 ID 都必须合法。各输出变量必须被 map section 恰好覆盖一次：赋予一个 unconditional mapping，或声明 DEFER。COPY_RANGE 等价于 n 个对应 COPY 定义，COPY_PAIRS 等价于各 pair 定义。这些记录不产生 FactId，不自动 discharge 任何 obligation。DEFER 初始化为 UNASSIGNED，只有 tape 中的 MAP 指令可以在其路径上定义；静态已定义变量不可重写。

4.8 Instruction stream

每条 instruction 的 header 是 8 B：

偏移	宽度	字段
0	1	`opcode:u8`
1	1	`flags:u8`
2	2	`aux:u16`
4	4	`payload_bytes:u32`

header 开始地址必须 8 B 对齐；opcode-specific payload 长度必须精确符合第三章，而不只是“至少足够”。`next_pc=align8(pc+8+payload_bytes)`，必须落在当前 block 范围内。padding 不参与 payload 长度计算。base opcode 以外的 byte 值，包括预留的 batch 与 theorem opcode，都返回 `E_UNSUPPORTED`。

4.8.1 隐式结果编号

producer payload 包含期望的 out ExprId，不包含目标 FactId。FactId 根据完整语法结构确定：普通 producer 占一个 slot；split 先按 case 顺序遍历所有 child 的定义，再占 `aux` 个 merged result slot。NOP、CHECK、MAP、DROP、RET、YIELD 与 CLOSE.FALSE 不产生 slot。格式合法但语义不可达的 instruction 仍占其语法 slot，不提交 live fact。该规则属于 P-03，确保跳过不可达语义不会改变后续 case 的编号。

4.8.2 Split 内部字节范围

BIN 与 SIGN 的固定 payload 为 16 B，THR 为 16 B。child byte length 包含该 child 的全部 instruction header、payload、instruction padding 与嵌套 split；必须是 8 的倍数且大于零。children 紧密连接，父 instruction 的 payload 必须恰好容纳固定字段和全部 child ranges。每个 child 都从独立的 8 B 边界开始，不允许跨 child 的 instruction 或引用本不存在的第三支。

RANGE 固定字段为 24 B，随后是 `branch_count` 个 u32 child length；该表末尾补零至 8 B 对齐，再连接所有 child。其 payload 长度必须满足

$$\mathrm{align}_8(24 + 4N) + \sum_{i=0}^{N-1} B_i.$$

其他二支 split 的 payload 长度为 $16 + B_0 + B_1$。全部值必须在 u32 payload 长度与实际文件边界内可表示。branch terminal、yield arity 和 proof state 的要求见执行章及指令章。

4.9 可选数据和版本边界

基础 v0.1 的 WITNESS 与 DEBUG 是不可信的 opaque byte section。它们不产生 fact，不提供额外 assumption，不定义 mapping，不 discharge obligation，也不能指定忽略错误。DEBUG 可以保存行号、符号和注释，具体 schema 尚未冻结；不影响数学判定。如果将来 WITNESS 含有证明必需的逻辑输入，它必须使用新定义的 mandatory section 或明确版本扩展。

v0.1 不忽略未来 minor version，不执行保留 opcode，也不接受未声明的 CALL checker。可选非逻辑 metadata 可以扩展；证明语义的扩展必须有显式的版本与能力约定。

第五章　Loader、验证结果与可信边界

5.1　输入与输出

一次验证请求包含 canonical source model、canonical target model、RPX 字节序列以及调用者要求的 contract。模型提供者必须给出已验证的模型结构和与其对应的 digest，或允许 loader 按已规定的 canonical 序列化重新计算 digest。仅有两个由不可信文件自行提供的 hash 字符串，不能证明它们绑定了调用者的模型。

verifier 必须返回明确的判定类别，而不把所有失败归并成“数学命题为假”。

结果	含义
`VERIFIED`	所需全部方向成功，contract 已证明。
`INVALID`	格式、引用、证明、映射或 obligation 违反规范。
`UNSUPPORTED`	文件版本、mandatory section、opcode 或能力未被本 verifier 支持。
`RESOURCE_LIMIT`	已公布的资源预算被超过；不表示证明错误。
`IO_ERROR`	无法取得完整稳定的文件或模型数据；不表示证明错误。

上述名称是接口草案，不是已经冻结的 CLI exit code ABI。任何非 VERIFIED 结果都不得被调用者解释为 reformulation 正确。

5.2　处理阶段

建议采用以下顺序；同一阶段内可使用等价的确定性检查顺序。不得在尚未确认 byte range 合法时读取其内容。

1. **取得稳定输入**。获取文件实际长度、只读字节视图和两个模型。在整个验证期间，已检查的字节与模型不得变化。
2. **检查 header**。验证 magic、版本、endianness、长度、reserved、directory offset 和 contract word。
3. **检查 directory**。对每个 entry 检查大小、alignment、count、存在性、重复与不相交范围。
4. **绑定模型与 contract**。比较 source/target digest，验证模型的 ID 范围与调用者要求；构造每个方向的完整 obligation 集合。
5. **验证 pool**。验证所有 ScalarPool 记录以及两个 expression pool 的 canonical 条件、引用范围和变量范围。
6. **验证逻辑 section 结构**。解码 map section；检查整条 tape 的 header、精确 payload 长度、flags、padding、嵌套范围和 terminal。计数全部 instruction 与 syntactic fact slot，包含不可达部分。
7. **执行 Forward**。绑定 Source 为 input、Target 为 output，应用静态 map，建立全新的方向状态，顺序验证 tape。
8. **执行 Reverse**。丢弃 Forward 的方向局部状态，重新绑定模型，使用独立 pool、mapping、facts 与 obligations。
9. **提交结果**。两个所需方向均成功后才报告 VERIFIED；可同时给出结构计数和运行统计。

结构验证和 proof execution 可以各扫描一次 tape。这不是 proof DAG 构建，也不需要保存每条

instruction 的对象。实现可以使用显式有界 frame stack 重复解码 header，而非依赖宿主函数递归。预验证层只确认候选字段和 byte range；数学推导与动态 live/scope 条件仍由执行层验证。

5.3 必需的结构检查

5.3.1 区间与整数

验证 $o+\ell$ 时必须先检测 overflow，再要求 $o+\ell \le B$。验证固定记录大小时必须检测 $c \times s$ 的 overflow，并要求等于声明 length。验证对齐时不能仅检测 file offset，还须检查每条 child instruction 的起点与终点。map range 的 `start+count`、RANGE 的 case count、FactId slot 总数也适用相同原则。

所有 section 起点必须位于 directory 末尾向上对齐后的 body 区域，即使其 length 为零。除基础格式允许的空 section 外，section 的字节必须能被对应 layout 精确消费。map 与 tape 的 declared count 必须与实际扫描计数相同。不得因提前 RET、false 或未知 metadata 而跳过 count 核验。

5.3.2 未执行区域

kernel 证明 scope 不可达后，不再检查该 scope 中后续 instruction 的数学效果与动态 fact 可用性。但整个文件的格式仍须有效：未知 opcode、越界 payload、错误 flags、非法 pool 和不完整 child range 仍拒绝。它们不能隐藏在死分支里。语法上的 producer 仍占其 FactId slot；它不会产生可以引用的 live fact。这种区分必须由实现明确保持，不能用一个未经检查的 byte skip 代替完整结构验证。

5.4 执行检查与提交原子性

一条 producer 必须先取得全部操作数、验证 kind 和作用域、执行精确算术、比较期望表达式，最后提交 live fact。不成功的 producer 不产生部分 fact。CHECK 在全部核验通过后才把对应 obligation 置为 DONE。MAP 在检查 output ID、DEFER 状态与 expression 合法后才写入 mapping。DROP range 必须先验证全部成员可释放，再进行释放；不得在中途错误时暴露一个看似继续有效的部分状态。

验证错误可以终止整个请求，故不要求 verifier 在错误后继续执行。原子性的用途是确保成功执行路径与错误诊断不受半完成 instruction 的影响。任何算术快路径必须保持同样的提交规则。

5.5 诊断类别

符号	典型原因
`E_FORMAT`	magic、长度、padding、reserved、alignment、重复或重叠 section、payload/count 不符。
`E_VERSION`	不支持的 major/minor、endianness 或 header size。
`E_UNSUPPORTED`	未知 mandatory section、未知/保留 opcode、未声明的扩展能力。
`E_HASH`	source 或 target digest 与模型绑定不符。
`E_CONTRACT`	非法 contract word、调用者要求不符、目标方向或关系不匹配。
`E_SCALAR`	ScalarRef 越界、非 canonical rational、零分母、非法符号或 magnitude、错误内联形式。
`E_EXPR`	重复/乱序变量、零 coefficient、非 canonical affine expression。
`E_REF`	ID 越界、非法 tag、不存在的 input side、future/dead/out-of-scope fact、不存在的 assumption。

符号	典型原因
E_KIND	instruction 要求的 LE/EQ/INT 类型不符。
E_SCOPE	祖先 fact 的局部 DROP、RET/YIELD/terminal 位于非法 scope。
E_BRANCH	case 不完整、yield arity 或 proposition 不一致、split 前提不符合规则。
E_PROOF	线性组合不等、LE 负 multiplier、BOX 上界失败、整数系数/取整/矛盾条件失败。
E_MAP	重复赋值、静态覆盖不完整、非法 DEFER 写入、混合已定义/未定义汇合、非法封存。
E_OBLIGATION	错误/重复 CHECK、代换不匹配、遗漏输出 domain/row/objective obligation。

错误应附带方向、绝对 file offset、opcode、scope 路径和具体 reason；DEBUG 行号仅可作为额外不可信展示信息。同一字节流可能违反多个条件，D0 不冻结“第一个错误”的全球优先级或数值 error code。错误类别的目的在于使拒绝原因可检查，不要求在一个无效文件上完成所有诊断。

5.6 资源预算

调用者可以设置最大文件大小、scalar 编码长度、算术 bit length、split 深度、case 数、live fact 数、临时累加器大小或运行时间。达到这些限制必须报告 RESOURCE_LIMIT，并指明限额。不得把算术 overflow 转换成饱和数、浮点近似或“先接受再复核”。格式字段本身规定的不可表示范围是 INVALID；实现额外公布的资源限制是 RESOURCE_LIMIT。

即时 31-bit integer 不意味着所有运算都限制在 31 bit。精确算术可以随证明增长；用有限整数快路径时，overflow 必须触发无损提升，或明确的资源失败。对非常大的合法 count，必须先核验 byte range 和资源限额，再申请容量或循环展开。

5.7 文件映射与稳定性

mmap 只提供字节视图；只读打开不保证验证期间文件不变。输入必须是稳定快照，可采用受控不可变文件、平台快照或自有不可变缓冲区；所有读取须来自同一份已验证数据。kernel 可依赖 canonical model 与不可变 byte slice，将操作系统映射约束留在 I/O 层；直接读入缓冲区同样符合规范。在 Rust 中，file-backed mapping 的安全前提由 I/O 层满足；memmap2 文档明确指出外部修改可破坏其安全假设。[1]

5.8 可信计算基与扩展

基础可信计算基包含：canonical model 的语义和绑定、边界检查、精确有理算术、core 指令 checker、scope/mapping/obligation 状态机，以及最终 contract 判定。assembler、证书生成器、solver、DEBUG、压缩工具、proof 优化工具均不自动属于可信逻辑。

将来加入 CALL_THM 时，必须显式给出 theorem ID、参数 schema、checker 版本和调用者允许列表。未知 theorem 不可解释为成功。基础 v0.1 对 opcode 0xF0 统一拒绝；不读取一个隐藏的“插件成功”标记。batch 扩展也必须定义完整展开与可观察状态，而非仅证明它最终得到某个相似的 fact。

[1] https://docs.rs/memmap2/latest/memmap2/struct.Mmap.html

第六章　RPA 文本汇编与工具链

RPA（Reformulation Proof Assembly）是 RPX 的文本表示。文本层承担符号解析、精确数值解析、表达式展开和二进制编码；证书是否成立仍由 RPX verifier 判定。本章定义用于人工编写、生成器输出和反汇编的基础语法，不赋予 assembler 任何独立的证明权力。

本章中的命令行和工具名称是 v0.1 工具接口草案。它们描述计划交付的行为，不表示当前仓库已经实现这些程序。MCM 的二进制编码和完整 MPS 导入覆盖范围尚待单独冻结。

6.1　文件结构

6.1.1　编码与词法约定

RPA 文件采用 UTF-8 编码，不含 BOM。语言关键字、标识符和操作数字符使用 ASCII；注释可以使用 UTF-8。关键字区分大小写，规范输出采用本章给出的小写拼写。空格和制表符是 token 分隔符；行结束可以是 LF 或 CRLF。assembler 在词法处理时将 CRLF 视为 LF。

；引入行注释，注释持续到该行结束。基础语法不支持块注释、宏、条件汇编、文件包含或用户定义伪指令。分号不是语句分隔符。除花括号中的列表外，一条语句必须在同一物理行结束；列表允许跨行。文件末尾可以省略最终换行。

标识符由字母或下划线起始，随后可以包含字母、数字和下划线。Fact 和 expression 标签另外允许全数字名称。这里的数字名称仍然是*符号名称*，不是用户指定的二进制 ID。例如作者写的 `%99` 可以被分配为第一个 derived FactId；规范反汇编则写成 `%0`。同一 execution domain 内，同类标签不得重复定义。两个 domain 的标签命名空间独立。

源码中的变量、行和静态事实索引必须是非负十进制整数。前导零允许被 assembler 接受，规范输出必须删除前导零。引用索引必须满足对应模型或 FactRef 编码的取值范围；表达式不得引用输出变量。

6.1.2　顶层声明

一个基础 v0.1 文件包含版本、两个模型摘要、contract 和两个 execution domain。`.forward` 和 `.reverse` 各出现一次，并按此顺序排列。每个 domain 恰好包含一个 `.map` 区域和一个 `.code` 区域；所有 `.expr` 声明位于 `.map` 之前。

Listing 6.1: RPA 文件结构；尖括号表示说明性占位符

```
.rpa 0.1
.source sha256 "<64 hexadecimal digits>"
.target sha256 "<64 hexadecimal digits>"
.contract mip=opt-equivalent
.contract forward.obj=eq
.contract reverse.obj=eq

.forward
.expr $name = <affine expression>
.map
    <static mapping declarations>
.endmap
.code
    <proof instructions>
    ret
.endcode

.reverse
.expr $other = <affine expression>
.map
    <static mapping declarations>
.endmap
.code
    <proof instructions>
    ret
.endcode
```

`.source` 和 `.target` 的字符串必须各包含恰好 64 个十六进制数字。输入可以使用大小写字母，规范输出使用小写。摘要是相应 canonical model 的 SHA-256，不是原始 MPS 文本的摘要。hash 字符串不支持转义或空白。

声明	基础 v0.1 取值与约束
`.rpa 0.1`	必须为第一条非注释语句；一个文件只出现一次。
`.contract mip=opt-equivalent`	要求双向可行映射与双向目标关系；每个方向的目标关系必须为 `eq` 或 `le`。
`.contract mip=feasibility-equivalent`	要求双向可行映射；允许 `none`、`eq` 或 `le`。指定 EQ/LE 时还必须完成对应目标义务。
`.contract forward.obj=eq`	Forward 的归一化目标差要求等于零。
`.contract forward.obj=le`	Forward 的归一化目标差要求不大于零。
`.contract forward.obj=none`	仅用于可行性等价 contract；不生成 Forward objective obligation。
`.contract reverse.obj=...`	与 Forward 相同，但 input/output 模型方向交换。

每条 contract 声明必须恰好出现一次，不能通过重复声明覆盖先前取值。Forward 和 Reverse 共同证明文件声明的 contract。调用者要求的最小 contract 由 verifier 的调用参数决定；文件不能自行降低调

用者要求。最大化目标和最小化目标的归一化定义见机器模型与 contract 章节。

6.2 精确数值字面量

6.2.1 允许的拼写

基础语法接受整数、分数、有限十进制数和十进制科学计数法。分数字面量的分母必须为严格正整数；符号只放在分子之前。

```
17
-3
4/7
-22/13
0.125
.125
1.
1e-6
-2.5E+3
```

这些写法均通过整数运算解析，不经过 IEEE 浮点数。十进制和科学计数法定义如下。将十进制尾数的全部数字拼接为整数 d，设小数点后有 k 位，指数为整数 t，符号为 $s \in \{-1, 1\}$，则：

$$\text{value} = s\, d\, 10^{t-k}.$$

例如 `0.125` 解析为 $125/1000 = 1/8$，`1e-6` 解析为 $1/1000000$，`-2.5E+3` 解析为 -2500。分数通过约分得到 canonical rational。`-0`、`0/7` 和 `0.0` 均归一化为整数零。

必须： assembler 必须以精确整数运算实现数值字面量转换，不能先解析为浮点数再尝试恢复有理数。

基础语法不接受 `NaN`、`Inf`、十六进制数、数字分组下划线、循环小数或隐含分母符号。`1/0` 是错误；`1/-2` 必须写为 `-1/2`。资源限额可以限制字面量长度和指数绝对值，但不得改变其数学值。

6.2.2 规范输出

规范反汇编只输出整数或约分后的 `p/q`：分母严格正，整数不带 `/1`，正数不带前导加号，零写作 `0`。因此源码中的十进制拼写、科学计数法、前导零和原始分数形式不会被保留。

6.3 操作数与符号

操作数形式	类别	意义
`%name`	Derived FactRef	当前 domain 已定义且可见的事实。
`@in.r[n].hi`	静态 FactRef	input row 的有限 upper side，类型为 LE。
`@in.r[n].lo`	静态 FactRef	input row 的有限 lower side，类型为 LE。
`@in.r[n].eq`	静态 FactRef	input equality row，类型为 EQ。
`@in.v[n].hi`	静态 FactRef	input variable 的有限 upper bound，类型为 LE。

操作数形式	类别	意义
`@in.v[n].lo`	静态 FactRef	input variable 的有限 lower bound，类型为 LE。
`@in.v[n].int`	静态 FactRef	input variable 的 integrality，类型为 INT。
`@case`	Branch FactRef	当前最内层分支假设；等价于 `@case[0]`。
`@case[n]`	Branch FactRef	距当前分支 n 层的祖先分支假设。
`$name`	ExprId 符号	当前 domain 的 canonical affine expression。
`@in.v[n]`	Input VarId	表达式或 mapping 中的 input variable。
`@out.v[n]`	Output VarId	mapping 或 variable obligation 的对象。
`@out.r[n]`	Output RowId	row obligation 的对象。

静态 FactRef 不产生指令，也不占用 derived FactId。只有模型确实声明的事实可以被引用：没有有限 upper bound 时，`@in.v[n].hi` 不存在；连续变量没有 `.int` 事实；非 equality row 没有 `.eq` 事实。equality row 的 side 可用性遵循 canonical model 对有限 sides 的定义。

`@case[n]` 中的 n 是 lexical scope depth offset，不是分支编号。当不存在该层假设时引用无效。`@case` 的具体类型和表达式由 split 指令规定，不能由源码重声明或赋值。

6.4 表达式声明与展开

6.4.1 仿射语法

表达式语法支持精确常数、input variable、此前声明的 expression 符号、括号、加减法及标量乘法：

```
.expr $x = @in.v[0]
.expr $sum = @in.v[0] + @in.v[2] - 1
.expr $scaled = 3/5 * $sum + 2
.expr $neg = -$scaled
.expr $mixed = 2 * (@in.v[0] - @in.v[2]) + 7
```

expression 符号通过展开得到 affine expression；符号不是运行时节点。引用必须指向同一 domain 内此前声明的 expression，因此展开关系无环。每个声明最终归一化为一个常数和严格按 VarId 排序的非零系数列表。

不允许 variable 与 variable 相乘、不允许两个一般表达式相乘，也不允许除以一般表达式。乘法的左侧必须是一个数值字面量。`3 * @in.v[0]` 合法；`@in.v[0] * 3` 不属于基础语法。减法、负号和括号可用于任意 affine expression。`2 @in.v[0]` 这样的隐含乘法不合法。

6.4.2 指令中的 expression 操作数

基础指令的 expression 操作数必须为 `$name`。例如 `map.aff @out.v[0], -$e` 不合法，必须先声明 `.expr $neg = -$e`，再使用 `map.aff @out.v[0], $neg`。这一限制确保 instruction 编码直接引用

expression pool，并让反汇编语法唯一。

assembler 可以合并数值相同的 expression 或 scalar pool 条目；它必须保留程序的数学语义和 scope 行为。基础文件格式是否允许重复 pool 条目由 wire format 章节规定，文本标签不承担 pool 唯一性保证。

6.5 静态 mapping 区域

`.map` 描述不依赖分支的 output variable mapping。它的变量命名方向随 domain 改变：Forward 中 input 是 Source、output 是 Target；Reverse 中相反。

```
.map
    copy.range @out.v[0], @in.v[0], 1000
    copy @out.v[1000], @in.v[7]
    const @out.v[1001], 0
    aff @out.v[1002], $replacement
    defer @out.v[1003]
    defer.range @out.v[1004], 4
.endmap
```

语句	文本操作数	section 编码与含义
`copy.range`	output 起点、input 起点、count	COPY_RANGE；连续 *count* 对变量作 identity-copy。
`copy`	output variable、input variable	COPY_PAIRS 的单对声明；assembler 可以合并多条声明。
`const`	output variable、number	CONST；output 等于该精确常数。
`aff`	output variable、expression	AFF；output 等于该 input affine expression。
`defer`	output variable	DEFER；该变量必须在 proof tape 的可达路径上定义。
`defer.range`	output 起点、count	连续 DEFER 声明的文本缩写。

必须： 静态 map 必须恰好覆盖全部 output variables；每个变量恰好由 COPY、CONST、AFF 或 DEFER 覆盖一次。

range 的 count 必须为严格正整数，索引范围的加法必须检查溢出。两个范围重叠、copy 越界或重复声明均是错误。一个静态 DEFER 标记不是已经完成的 mapping，也不能消除任何 domain obligation。

`map.copy`、`map.const` 和 `map.aff` 是 proof tape 指令，语法与相应静态声明相近。它们只用于当前路径尚未赋值的 DEFER output variable。对已有 mapping 重复赋值不合法。不同 sibling branches 可以各自给同一个 DEFER variable 赋值，因为两个分支分别从父状态执行。

6.6 基础指令的文本形式

下列语法穷尽基础 v0.1 的 28 个 core opcodes。所有指令的数学检查、异常条件和二进制 payload 见 ISA 章节；本节只规定文本 spelling 和操作数顺序。

6.6.1 General、linear 与 integer 指令

```
nop
ret
drop %fact
drop %first .. %last

%out = lin.le $expr {
    +1 @in.r[0].hi
    -3/5 @in.r[1].eq
}
%out = lin.eq $expr {
    +1 @in.r[0].eq
    -2 %earlier
}
%out = eq.pair $expr, %positive, %negative
%out = box.le $expr
%out = weaken.le $expr, %input, 3/2
close.false %contradiction

%out = int.aff $expr, 7 {
    +3 @in.v[0].int
    -2 %integer_fact
    +1/2 %equality_fact
}
%out = round.hi $expr, %upper, %integer
%out = round.lo $expr, %lower, %integer
```

线性列表中的每一行恰好包含一个 multiplier 和一个 FactRef, 中间至少一个空白, 不加逗号。允许空列表; 空 `lin.eq` 或 `lin.le` 只能证明零表达式。`int.aff` 的 offset 必须是精确整数。整数事实的 multiplier 必须为整数; EQ 项的 multiplier 可以是任意有理数, 这是整数性通过等式传递的草案修订 P-04。

`drop` 的区间端点经符号解析后必须形成递增的连续 FactId 区间, 其中每个 ID 均为当前 scope 已定义的 live fact。区间包含未来、dead 或其他 scope 的事实时不合法; 不能因文本标签恰好具有连续数字而假定其机器 ID 连续。assembler 可以将 liveness 结果拆成多条合法 DROP 区间。

6.6.2 Mapping 与 obligation 指令

```
map.copy @out.v[0], @in.v[1]
map.const @out.v[1], 0
map.aff @out.v[2], $expr

check.row.hi @out.r[0], %fact
check.row.lo @out.r[0], @in.r[1].lo
check.row.eq @out.r[0], %fact
check.var.hi @out.v[0], %fact
check.var.lo @out.v[0], @in.v[1].lo
check.var.int @out.v[0], %fact
check.obj.eq %fact
check.obj.le %fact
```

CHECK 的 proof operand 可以是任意合法 FactRef，包括静态模型事实和 branch assumption，无需先复制到 derived register。其类型和表达式必须与 kernel 生成的 obligation 精确一致。

6.6.3 `exact` 简写

修订提案 P-05。为 CHECK 增加 `exact` 文本操作数；编译为 CHECK 的 EXACT flag，并由 kernel 直接检查下述有限规则。它不是 assembler 信任断言。

```
check.row.eq @out.r[0], exact
check.var.int @out.v[0], exact
check.obj.eq exact
```

exact 的判断对象始终是 kernel 自己由 output model 和当前 mapping 构造的表达式 e：

所需 FactKind	EXACT 可接受条件
EQ	canonical e 为恒等零。
LE	canonical e 无变量项，且其常数不大于零。
INT	e 的常数和所有系数均为整数；所有非零项涉及的 input variables 在 input model 中声明为整数或 binary。

EXACT 不搜索 input rows，不自动匹配 input bounds，也不推导未知变量的整数性。相同 input row 直接用于 output row 时应显式写出对应静态 FactRef。一个必须依赖已验证 INT fact 的整数性检查，应使用该 fact 或 `int.aff`，不能要求 EXACT 发现依赖。

模型相关的 `exact` 预检查和 expression 归一化要求 assembler 获得两个 canonical models。即使 assembler 已预检查成功，verifier 仍必须独立执行同一判定；源文本不能成为可信输入。

6.6.4 Structured cases

```
(%a, %b) = split.bin @in.v[0] {
case 0:
    yield %left_a, %left_b
case 1:
    yield %right_a, %right_b
}

(%a) = split.int.thr %integer, 13 {
case le:
    yield %left
case ge:
    yield %right
}

(%a) = split.int.range %integer, %lower, %upper, 3, 5 {
case 3:
    yield %value3
case 4:
    yield %value4
case 5:
    yield %value5
}

() = split.sign $expr {
case le:
    yield
case ge:
    yield
}
```

split 总是带输出 tuple；零个输出使用 `()`。`split.bin` 的 cases 必须按 `0`、`1` 顺序出现；`split.int.thr` 和 `split.sign` 的 cases 必须按 `le`、`ge` 顺序出现；`split.int.range` 的 cases 必须按闭区间中的整数递增排列。case labels 不是可跳转地址。

所有 sibling branches 必须从同一父状态开始执行。若一个 DEFER variable 在部分可达 branches 中已赋值、另一些可达 branches 中仍未赋值，merge 立即返回 `E_MAP`。只有所有可达 branches 均未赋值时才保留 UNASSIGNED；均已赋值时按 mapping merge 规则保留共同 expression 或封存为 SEALED_PIECEWISE。

每个 case 必须以 `yield` 或 `close.false` 结束，之后不能有指令。`yield` 的 FactRef 数量必须等于 split 输出 tuple 长度；`yield` 不带操作数表示零个事实。所有可达 branches 在每个 yield 位置必须产生完全相同的 canonical proposition。已证明不可达的 branches 不需要 yield。split 可以嵌套，但不允许 JMP 或其他非结构化跳转。

`split.int.thr` 的 threshold 和 `split.int.range` 的两个端点必须为精确整数。对于 INT expression g，range 所用 lower fact 必须精确表示 $-g + L \leq 0$，upper fact 必须精确表示 $g - U \leq 0$。闭区间端点要求 $L \leq U$。

对于 `split.sign`，两个 proof branches 的假设分别为 $e \leq 0$ 和 $-e \leq 0$，零点属于两支。mapping 的

数值应用若遇到零，按草案 P-06 选择先出现的 `le` case；验证仍检查两个 proof branches。

6.7 FactId、scope 与规范反汇编

修订提案 P-03。derived FactId 按语法顺序隐式分配：普通 producer 各分配一个 ID；split 先递归分配各 branch 的 producer IDs，再分配 merge outputs。ID 分配不依赖数学可达性。

assembler 先解析全部符号和结构，依此分配 FactId；kernel 仍禁止未来事实引用。split 输出在源码头部命名，但只有 split 完成后可见，不能在其 branches 内引用。branch-local facts 不能通过符号泄漏到其他 sibling branch 或父 scope。源代码在文本上较早定义某个 fact，不表示该 fact 在另一个 scope 中可见。

规范反汇编将 expression 符号重命名为 `$e0`、`$e1` 等，将 derived facts 重命名为机器 FactId 的十进制形式。它输出固定 case 顺序、约分数值、展开后的 canonical expressions 和确定的空白规则。

必须：assemble–disassemble–assemble 必须保持 contract、数学 propositions、控制结构和 mapping 语义；基础 v0.1 不承诺逐字节还原原始文件。

原因包括 scalar/expression pool 的合法重复条目、section 排列、debug 数据以及文本标签可能不同。若需要逐字节保存，工具必须提供独立的原始二进制导出模式，不能把这种需求隐藏在 canonical RPA 语义中。

6.8 核心 EBNF

以下 EBNF 使用 `[...]` 表示可选部分、`{...}` 表示重复、`|` 表示选择。`NL` 表示换行，`WS` 表示至少一个水平空白。注释在语法分析前处理；空白行允许出现在任意完整语句之间。表达式加减运算左结合；scalar 乘法优先于加减法，单目符号优先于加减法。

Listing 6.2: 词法与顶层语法

```
digit       = "0" | "1" | ... | "9" ;
letter      = "A" | ... | "Z" | "a" | ... | "z" ;
hex-digit   = digit | "A" | ... | "F" | "a" | ... | "f" ;
WS          = (" " | TAB), { " " | TAB } ;
NL          = LF ;
uint        = digit, { digit } ;
sign        = "+" | "-" ;
integer     = [ sign ], uint ;
fraction    = [ sign ], uint, "/", uint ;
decimal     = uint, ".", [ uint ] | ".", uint ;
mantissa    = uint | decimal ;
scientific  = [ sign ], mantissa, ("e" | "E"), integer ;
number      = fraction | scientific | [ sign ], decimal | integer ;
identifier  = (letter | "_"), { letter | digit | "_" } ;
name        = identifier | uint ;
fact-name   = "%", name ;
expr-name   = "$", name ;
input-var   = "@in.v[", uint, "]" ;
output-var  = "@out.v[", uint, "]" ;
output-row  = "@out.r[", uint, "]" ;
fact-ref    = fact-name
            | "@in.r[", uint, "].", ("hi" | "lo" | "eq")
            | "@in.v[", uint, "].", ("hi" | "lo" | "int")
            | "@case", [ "[", uint, "]" ] ;
hash        = '"', 64 * hex-digit, '"' ;
objective   = "eq" | "le" | "none" ;
contract    = ".contract mip=", ("opt-equivalent"
              | "feasibility-equivalent"), NL,
              ".contract forward.obj=", objective, NL,
              ".contract reverse.obj=", objective, NL ;
file        = ".rpa 0.1", NL,
              ".source sha256", WS, hash, NL,
              ".target sha256", WS, hash, NL,
              contract, forward-domain, reverse-domain ;
forward-domain = ".forward", NL, domain-body ;
reverse-domain = ".reverse", NL, domain-body ;
domain-body = { expression-declaration },
              ".map", NL, { map-declaration }, ".endmap", NL,
              ".code", NL, code-body, ".endcode", [ NL ] ;
```

Listing 6.3: 仿射表达式与静态 mapping 语法

```
expression-declaration = ".expr", WS, expr-name, "=", affine, NL ;
affine      = scaled, { ("+" | "-"), scaled } ;
scaled      = number, "*", signed-primary | signed-primary ;
signed-primary = [ sign ], primary ;
primary     = number | input-var | expr-name | "(", affine, ")" ;

map-declaration = "copy.range", WS, output-var, ",", input-var,
                    ",", uint, NL
                | "copy", WS, output-var, ",", input-var, NL
                | "const", WS, output-var, ",", number, NL
                | "aff", WS, output-var, ",", expr-name, NL
                | "defer", WS, output-var, NL
                | "defer.range", WS, output-var, ",", uint, NL ;
```

Listing 6.4: 普通指令语法

```
linear-list = "{", NL, { number, WS, fact-ref, NL }, "}" ;
proof       = fact-ref | "exact" ;
instruction = "nop"
            | "drop", WS, fact-name, [ "..", fact-name ]
            | fact-name, "=", ("lin.le" | "lin.eq"), WS,
                expr-name, linear-list
            | fact-name, "=", "eq.pair", WS, expr-name, ",",
                fact-ref, ",", fact-ref
            | fact-name, "=", "box.le", WS, expr-name
            | fact-name, "=", "weaken.le", WS, expr-name, ",",
                fact-ref, ",", number
            | fact-name, "=", "int.aff", WS, expr-name, ",",
                number, linear-list
            | fact-name, "=", ("round.hi" | "round.lo"), WS,
                expr-name, ",", fact-ref, ",", fact-ref
            | "map.copy", WS, output-var, ",", input-var
            | "map.const", WS, output-var, ",", number
            | "map.aff", WS, output-var, ",", expr-name
            | ("check.row.hi" | "check.row.lo" | "check.row.eq"),
                WS, output-row, ",", proof
            | ("check.var.hi" | "check.var.lo" | "check.var.int"),
                WS, output-var, ",", proof
            | ("check.obj.eq" | "check.obj.le"), WS, proof
            | split ;
```

Listing 6.5: 终止、split 与 case 语法

```
fact-list   = fact-ref, { ",", fact-ref } ;
output-list = fact-name, { ",", fact-name } ;
yield       = "yield", [ WS, fact-list ] ;
false       = "close.false", WS, fact-ref ;
branch-body = { instruction, NL }, (yield | false), NL ;
code-body   = { instruction, NL }, ("ret" | false), NL ;
split       = "(", [ output-list ], ")", "=", split-op ;
split-op    = "split.bin", WS, input-var, "{", NL,
                "case 0:", NL, branch-body,
                "case 1:", NL, branch-body, "}"
            | "split.int.thr", WS, fact-ref, ",", number, "{", NL,
                "case le:", NL, branch-body,
                "case ge:", NL, branch-body, "}"
            | "split.int.range", WS, fact-ref, ",", fact-ref,
                ",", fact-ref, ",", number, ",", number, "{", NL,
                range-cases, "}"
            | "split.sign", WS, expr-name, "{", NL,
                "case le:", NL, branch-body,
                "case ge:", NL, branch-body, "}" ;
range-cases = "case", WS, integer, ":", NL, branch-body,
              { "case", WS, integer, ":", NL, branch-body } ;
```

该 EBNF 规定结构；类型、范围、case 完整性、scope 可见性和终止可达性属于静态及执行语义检查。token 之间可插入水平空白，除显式 `WS` 外不强制数量。线性列表的右花括号后由外围语句换行结束。assembler 必须报告源码位置，不能通过自动补全缺失 case 或 obligation 使不完整程序成为有效证书。

6.9 工具链接口草案

工具	接口职责
`rpa-as`	解析文本，读取 canonical models 以检查索引和 exact 简写，展开 expressions，解析 FactId 和 scopes，输出 RPX。允许实现 liveness 优化，但不得改变命题或丢掉 obligation。
`rpa-dis`	先验证 RPX 的结构，再输出 canonical RPA；不把成功反汇编解释为证明通过。
`rpa-objdump`	显示 header、section directory、opcode counts、scope 深度及 fact liveness 等结构信息。结构统计与验证运行统计必须明确区分。
`rp-verify`	读取两个 canonical models，绑定摘要和调用者要求，执行两个 proof tapes，成功后报告 VERIFIED 及 contract。
`mps-canon`	将明确支持的 MPS 子集转换为 canonical model；不支持的输入特性必须显式拒绝。

Listing 6.6: 计划中的模型相关汇编与验证接口

```
mps-canon source.mps -o source.mcm
mps-canon target.mps -o target.mcm

rpa-as --source source.mcm --target target.mcm \
    proof.rpa -o proof.rpx

rp-verify --require opt-equivalent \
    source.mcm target.mcm proof.rpx

rpa-dis proof.rpx -o proof.canonical.rpa
rpa-objdump --sections --opcodes --liveness proof.rpx
```

以上命令展示接口契约而非可立即运行的实现。verifier 的成功输出至少应包含 VERIFIED、实际检查的 contract 和两个 model 摘要。instruction 数、derived fact 数、peak live facts 和耗时应说明其统计定义，特别区分 proof tape 解码、exact arithmetic 和模型加载成本。任何失败必须返回非零退出状态，并给出失败 domain、指令位置、错误类别和简短原因。

6.10 扩展边界与保留语法

基础 v0.1 不接受 `.batch` 或 `call.thm`。批量编码区 `0x80--0xBF` 和 `CALL_THM` 的 `0xF0` 仅为未来扩展保留；基础 loader 遇到它们必须返回 `E_UNSUPPORTED`。

未来 batch 扩展必须定义完整的 core instruction 展开，包括 FactId 分配、scope、错误行为和 obligation 更新，不能通过批处理增加可证明命题。未来 theorem 扩展必须显式标识 trusted checker 和 TheoremId，不能用新名字绕过 trusted semantics 的版本审查。

第七章　完整证书示例与验收案例

本章给出三份完整的双向 RPA 程序，覆盖冗余约束删除、等式变量消元和分段 mapping。每个例子明确列出模型、变量 domains、目标和全部 obligations。所有模型目标均为最小化；例中没有被省略的模型行或变量。

源码中的 64 个十六进制字符摘要均为待替换占位值：Source 使用全零，Target 使用全一。它们具有合法词法形式，但不代表例中模型的真实 SHA-256。生成 RPX 前，必须使用 canonical model 的实际摘要替换；否则正确 verifier 应在模型绑定阶段返回 `E_HASH`。除摘要占位值外，程序不给出省略号或未定义符号。

本章使用的 `exact` 简写依赖 P-05，第三个例子的 mapping 封存依赖 P-02。它们是本手册明确提出的 v0.1 草案修订，应在冻结版本前一并审定。

7.1　示例 A：删除冗余上界行

7.1.1　模型与 contract

Source 和 Target 均只有一个连续变量，目标恒为零。Source 的第二行重复一个更松的上界。删除它不会改变可行域。

项	Source	Target
变量	$x_0 \in \mathbb{R}, \quad 0 \le x_0 \le 1$	$y_0 \in \mathbb{R}, \quad 0 \le y_0 \le 1$
row 0	$x_0 \le 1$	$y_0 \le 1$
row 1	$x_0 \le 2$	无
目标	$f_S(x) = 0$	$f_T(y) = 0$
Forward	$y_0 = x_0$	
Reverse	$x_0 = y_0$	

两个方向均证明目标等式。Forward 的 variable bounds 和 row 0 直接使用 input 静态事实；Reverse 中 Source row 1 用 `weaken.le` 从 $y_0 - 1 \le 0$ 得到 $y_0 - 2 \le 0$。

7.1.2 完整 RPA

Listing 7.1: 冗余行删除：双向完整程序

```
.rpa 0.1
.source sha256 "0000000000000000000000000000000000000000000000000000000000000000"
.target sha256 "1111111111111111111111111111111111111111111111111111111111111111"
.contract mip=opt-equivalent
.contract forward.obj=eq
.contract reverse.obj=eq

.forward
.map
    copy @out.v[0], @in.v[0]
.endmap
.code
    check.var.lo @out.v[0], @in.v[0].lo
    check.var.hi @out.v[0], @in.v[0].hi
    check.row.hi @out.r[0], @in.r[0].hi
    check.obj.eq exact
    ret
.endcode

.reverse
.expr $loose_upper = @in.v[0] - 2
.map
    copy @out.v[0], @in.v[0]
.endmap
.code
    check.var.lo @out.v[0], @in.v[0].lo
    check.var.hi @out.v[0], @in.v[0].hi
    check.row.hi @out.r[0], @in.r[0].hi
    %loose = weaken.le $loose_upper, @in.r[0].hi, 1
    check.row.hi @out.r[1], %loose
    check.obj.eq exact
    ret
.endcode
```

7.1.3 检查要点

Forward 生成并完成四个 obligations：两个 variable bounds、一个 row upper side、一个 objective equality。Reverse 完成五个：两个 variable bounds、两个 row upper sides 和一个 objective equality。连续变量不生成 integrality obligation；这些单侧行也不生成 lower-side obligation。

Reverse 的 derived fact 是：

$$(y_0 - 1) - 1 = y_0 - 2 \leq 0.$$

两个方向目标差均为恒等零，故 EXACT 的 EQ 规则成立。任何方向删除任一 CHECK 都应导致 RET 返回 `E_OBLIGATION`，即使该行从模型语义上看起来显然冗余。

7.2 示例 B：利用等式消去一个变量

7.2.1 模型与 contract

Source 有两个连续变量和一个等式；Target 保留第二个变量，并把目标中的第一个变量代换掉。

项	Source	Target
变量 0	$x_0 \in \mathbb{R}, \quad 0 \le x_0 \le 1$	$y_0 \in \mathbb{R}, \quad 0 \le y_0 \le 1$
变量 1	$x_1 \in \mathbb{R}, \quad 0 \le x_1 \le 1$	无
row 0	$x_0 + x_1 = 1$	无
目标	$f_S(x) = x_0$	$f_T(y) = 1 - y_0$
Forward	$y_0 = x_1$	
Reverse	$x_0 = 1 - y_0, \quad x_1 = y_0$	

Forward objective difference 不是 input 空间中的恒等零，而是 Source equality 的负值：

$$D_F(x) = (1 - x_1) - x_0 = -(x_0 + x_1 - 1).$$

因此 Forward 必须提供 EQ 事实，不能写成 `check.obj.eq exact`。Reverse 中代换后的 row equality 和 objective difference 则均为恒等零。

7.2.2 完整 RPA

Listing 7.2: 等式消元与反向重建：双向完整程序

```
.rpa 0.1
.source sha256 "0000000000000000000000000000000000000000000000000000000000000000"
.target sha256 "1111111111111111111111111111111111111111111111111111111111111111"
.contract mip=opt-equivalent
.contract forward.obj=eq
.contract reverse.obj=eq

.forward
.expr $objective_difference = 1 - @in.v[1] - @in.v[0]
.map
    copy @out.v[0], @in.v[1]
.endmap
.code
    check.var.lo @out.v[0], @in.v[1].lo
    check.var.hi @out.v[0], @in.v[1].hi
    %objective = lin.eq $objective_difference {
        -1 @in.r[0].eq
    }
    check.obj.eq %objective
    ret
.endcode

.reverse
.expr $complement = 1 - @in.v[0]
.map
    aff @out.v[0], $complement
    copy @out.v[1], @in.v[0]
.endmap
.code
    check.var.lo @out.v[0], @in.v[0].hi
    check.var.hi @out.v[0], @in.v[0].lo
    check.var.lo @out.v[1], @in.v[0].lo
    check.var.hi @out.v[1], @in.v[0].hi
    check.row.eq @out.r[0], exact
    check.obj.eq exact
    ret
.endcode
```

7.2.3 检查要点

Forward 的三个 obligations 是 Target 的两个 bounds 和 objective equality。Reverse 的六个 obligations 是 Source 的四个 bounds、一个 equality row 和 objective equality。canonical equality row 在这个 profile 中生成一个 EQ obligation，不能要求作者重复证明相同 row 的两条 LE sides。

Reverse 中四个 bounds 的归一化表达式依次为：

$$x_0 \geq 0 : -(1 - y_0) = y_0 - 1,$$
$$x_0 \leq 1 : (1 - y_0) - 1 = -y_0,$$
$$x_1 \geq 0 : -y_0,$$
$$x_1 \leq 1 : y_0 - 1.$$

它们分别精确匹配 input variable 的 upper、lower、lower、upper facts。目标常数 1 参与检查；漏掉该常数将使 objective relation 不成立。

此例也展示模型中的 equality 与恒等式的区别：Forward 使用 input equality，Reverse 使用 substitution identity。EXACT 不自动从 input equality 推理，避免把模型相关证明搜索隐藏进简写。

7.3 示例 C：分段 absolute-value mapping

7.3.1 模型与 contract

Source 和 Target 没有 rows，且目标均为零。Forward 使用 absolute-value mapping；Reverse 选择非负代表元。两个映射不互为逆函数，仍能证明双向可行性和目标值相同。

项	Source	Target
变量	$x_0 \in \mathbb{R}, \quad -1 \leq x_0 \leq 1$	$y_0 \in \mathbb{R}, \quad 0 \leq y_0 \leq 1$
rows	无	无
目标	$f_S(x) = 0$	$f_T(y) = 0$
Forward	$y_0 = \lvert x_0 \rvert$	
Reverse	$x_0 = y_0$	

Forward 的静态 map 只声明 DEFER。`split.sign` 的两支分别定义 $y_0 = -x_0$ 和 $y_0 = x_0$，并在分支内完成所有 bounds 及 objective obligations。它没有需要汇合的 Fact outputs，因此使用空 tuple 和空 YIELD。

7.3.2 完整 RPA

Listing 7.3: 分段映射与 scope 封存：双向完整程序

```
.rpa 0.1
.source sha256 "0000000000000000000000000000000000000000000000000000000000000000"
.target sha256 "1111111111111111111111111111111111111111111111111111111111111111"
.contract mip=opt-equivalent
.contract forward.obj=eq
.contract reverse.obj=eq

.forward
.expr $x = @in.v[0]
.expr $negative_x = -@in.v[0]
.map
    defer @out.v[0]
.endmap
.code
    () = split.sign $x {
    case le:
        map.aff @out.v[0], $negative_x
        check.var.lo @out.v[0], @case
        check.var.hi @out.v[0], @in.v[0].lo
        check.obj.eq exact
        yield
    case ge:
        map.aff @out.v[0], $x
        check.var.lo @out.v[0], @case
        check.var.hi @out.v[0], @in.v[0].hi
        check.obj.eq exact
        yield
    }
    ret
.endcode

.reverse
.expr $source_lower = -@in.v[0] - 1
.map
    copy @out.v[0], @in.v[0]
.endmap
.code
    %lower = weaken.le $source_lower, @in.v[0].lo, 1
    check.var.lo @out.v[0], %lower
    check.var.hi @out.v[0], @in.v[0].hi
    check.obj.eq exact
    ret
.endcode
```

7.3.3 检查要点与 mapping 封存

在 `le` branch 中，assumption 为 $x_0 \le 0$，恰好证明 output lower bound $-y_0 = x_0 \le 0$；Source lower bound 为 $-x_0 - 1 \le 0$，恰好证明 output upper bound $y_0 - 1 = -x_0 - 1 \le 0$。

在 `ge` branch 中，assumption 为 $-x_0 \le 0$，恰好证明 output lower bound；Source upper bound $x_0 - 1 \le 0$ 恰好证明 output upper bound。两支均完成同一组三个 obligations。merge 对可达 branches 的 DONE 位图取交集，故三个 obligations 在父 scope 中均完成。

两个 mapping expressions 一般不同，因此父 scope 不保留某个共同 affine expression；它把该 variable 标记为 `SEALED_PIECEWISE`。P-02 规定：该状态证明 mapping 在所有可达分支上完整，且所有依赖该变量的 obligations 已完成，因而 RET 可以成功。merge 之后再尝试使用它做新的 affine substitution 必须返回 `E_MAP`。

Source 点 $x_0 = 0$ 同时满足两个 proof guards。验证执行两个分支；数值 mapping 应用按 P-06 选择 `le`，两支在本例均给出 $y_0 = 0$。这种优先级是 mapping 应用规则，不允许少验证另一支。

Reverse 的 lower-bound fact 是 $(-y_0) - 1 \le 0$，即 $x_0 \ge -1$；upper bound 则直接继承 $y_0 \le 1$。Reverse 完成三个 obligations。Forward 每个分支完成三个 obligations，父 scope 的最终集合仍是三个，而不是把分支 CHECK 次数加成六个独立模型义务。

7.4 整数性与取整的小型推导片段

以下是用于指令级测试的局部片段，不是独立双向 certificate。假设 input 中 x_0 连续、x_1 整数，并有 equality row $x_0 - x_1 = 0$。草案 P-04 允许从整数变量和 equality 推出 x_0 的整数性：

```
.expr $x = @in.v[0]

%integer_x = int.aff $x, 0 {
    +1 @in.v[1].int
    +1 @in.r[0].eq
}
```

核验的 affine sum 为 $x_1 + (x_0 - x_1) = x_0$。EQ 项的有理数 multiplier 不改变整数性，因为 EQ expression 恒等于零。

若已经有 `%upper` 表示 $x_0 - 37/5 \le 0$，且 `%integer_x` 表示 $x_0 \in \mathbb{Z}$，则声明 `.expr $rounded = @in.v[0] - 7` 后可以使用：

```
%rounded = round.hi $rounded, %upper, %integer_x
```

ROUND.HI 的数值核验使用精确 floor，而不是向零截断。应补充负阈值验收用例：$x \le -7/5$ 应推出 $x \le -2$；ROUND.LO 中 $x \ge -7/5$ 应推出 $x \ge -1$。

7.5 LIN.LE 的二进制 golden vector

下列 encoding 对应一个 `lin.le` producer，输出 ExprId 为 5，组合 input equality row 0 与 input variable 0 的 lower bound，两个 multiplier 均为 +1。目的 FactId 隐式分配，不出现在 payload 中。

```
%0 = lin.le $e5 {
    +1 @in.r[0].eq
    +1 @in.v[0].lo
}
```

字段	值	Little-endian 字节
opcode	0x10	10
flags	0x00	00
aux	2	02 00
payload_bytes	20	14 00 00 00
out_expr_id	5	05 00 00 00
FactRef 0	0x30000000	00 00 00 30
ScalarRef 0	0x80000001	01 00 00 80
FactRef 1	0x50000000	00 00 00 50
ScalarRef 1	0x80000001	01 00 00 80
padding	4 个零字节	00 00 00 00

整条 instruction 占 32 bytes：8-byte header、20-byte payload 和 4-byte padding。

Listing 7.4: 完整 32-byte golden vector

```
10 00 02 00 14 00 00 00
05 00 00 00
00 00 00 30 01 00 00 80
00 00 00 50 01 00 00 80
00 00 00 00
```

这些字节只说明结构，不独立证明任何模型命题。ExprId 5 必须存在，并且其表达式必须精确等于对应 input equality expression 与 lower-bound expression 的和。改变模型或 expression pool 可使同样的 instruction 字节返回 `E_PROOF`。

7.6 失败证书与一致性验收

以下用例用于未来实现的验收清单。格式 loader、状态机和数学 checker 可以在不同阶段发现问题；错误代码表示规范要求的主要类别，不强制内部调用栈或错误文本。

变异或错误	错误类别	必须拒绝的原因
在 fact 定义前使用其 label	`E_REF`	对应机器引用为 future FactId。
DROP 后引用同一 fact	`E_REF`	dead fact 不得重新读取。
从 sibling branch 读取局部 fact	`E_REF`	事实未提升到共同父 scope。
在 LIN.LE 中给 LE 输入负 multiplier	`E_PROOF`	不符合非负 inequality 组合规则。
LIN.EQ 引用 LE 或 INT	`E_KIND`	等式线性组合只接受 EQ 输入。
改变 golden vector 的 output expression	`E_PROOF`	指定表达式与精确组合不相等。
缺少 output variable 的 map 或 DEFER 覆盖	`E_MAP`	静态 mapping 未完整覆盖输出变量。

变异或错误	错误类别	必须拒绝的原因
同一路径重复赋值 output variable	E_MAP	mapping 不允许覆盖已有定义。
删去示例 A 的任意 row CHECK	E_OBLIGATION	RET 发现未完成的 output row。
示例 B Forward 将 objective proof 换为 EXACT	E_PROOF	目标差是 input equality 的负值，而非恒等零。
示例 C 只有一支完成 bounds	E_OBLIGATION	obligation merge 必须对可达 branches 取交集。
split.bin 缺少 case 1	E_BRANCH	分支不满足规定的 exhaustive 结构。
branch 在 YIELD 后含有指令	E_BRANCH	terminator 必须是 branch 的最后一条指令。
两支 YIELD 的 proposition 不同	E_BRANCH	merge 不能产生共同事实。
CONST scalar 的分母为零	E_SCALAR	不是有理数；不能进入执行。
表达式变量重复或系数为零	E_EXPR	wire expression 未满足 canonical 结构。
header 或 section 越界、非法重叠	E_FORMAT	文件结构不可安全读取。
opcode 为未实现的保留扩展	E_UNSUPPORTED	基础 v0.1 不能忽略未知指令。
实际模型摘要与 header 不同	E_HASH	proof 未绑定到调用者提供的模型。
文件只证明可行性，调用者要求最优等价	E_CONTRACT	文件不能降低调用者要求。

还应为每份成功证书建立对应的边界失败案例：非 canonical rational、最小及最大 ScalarRef immediate、64-bit offset overflow、零-length branch、过深 nesting、DROP 穿越 scope、INT threshold 的负数和 RANGE 端点相反等。资源限额触发与逻辑 REJECT 应分别报告；限额不能把未经完整验证的证书报告为 VERIFIED。

本章示例应在实现阶段成为模型 fixtures、assembler round-trip fixtures 和 verifier acceptance fixtures。它们的意义是固定可观察语义，而非替某个内部数据结构规定测试细节。

第八章　执行成本、实现建议与测量

8.1　成本模型的边界

本章是说明性内容，定义测量方法而不承诺未实现系统的性能。令 B 为 RPX 字节数，W_{lin} 为线性推导所访问的 coefficient 项总数，W_{sub} 为 mapping 代换访问的项总数，C_{exact} 为所有精确算术的 bit complexity。合理的目标模型是

$$T = O(B + W_{\text{lin}} + W_{\text{sub}} + C_{\text{exact}} + W_{\text{state}}).$$

W_{state} 包含 map 初始化、obligation 初始化与分支状态回滚/汇合。如果一次请求还需从文件验证两个模型并计算 digest，还须计入 $|M_S| + |M_T|$ 的读取与校验成本。已验证的稳定模型缓存可以让后续请求复用这部分工作；缓存键必须绑定真实 canonical 数据。

“线性扫描 tape” 不表示全部数学工作都与 instruction 数成正比。一条包含很长 sparse row 的 LIN 或 CHECK 可以比许多短 instruction 更昂贵。完整结构扫描和语义扫描各执行一次仍为 $O(B)$；无需为每条指令建立 DAG node。

8.2　代换成本

对于输出 row r 和 mappings $y_j = E_j(x)$，代换为

$$A_r E(x) - u_r = c_r + \sum_{j \in \text{supp}(A_r)} a_{rj} E_j(x).$$

一次直接展开访问的 terms 数由

$$\sum_{j \in \text{supp}(A_r)} \text{nnz}(E_j)$$

控制，而不是仅由 $\text{nnz}(A_r)$ 控制。不同 E_j 之间的变量可以重合、相消，也可能使结果显著变稠密。相同 row 的重复代换还会产生重复读取成本。

稳定的 unconditional mapping 可以缓存其代换结果或保持 COPY 引用路径。分支内的 mapping 状态必须参与 cache 身份，退出和回滚时不能保留错误路径的结果。SEALED_PIECEWISE 不允许后续代换，因此不会要求缓存一般 piecewise AST。

8.3　精确算术与累加器

建议对机器整数或小有理数使用 checked arithmetic，对不能表示的中间值无损提升到大整数。ScalarRef 的内联编码只减少文件和解码开销；它本身不规定算术对象，也不自动避免 BigRational 分配。目标 expression 已由证书提供，kernel 可以计算 residual 并检查所有系数与 constant 恰为零。

可选择的精确累加器包括 sorted sparse merge、hash accumulator 或带 touched 集合的 dense accumulator。选择依赖模型维度、每条推导访问项数和 coefficient 分布。使用 dense 工作区时，建议只清理 touched entry，避免每条 instruction 都遍历全部 input variables。证书已经携带 canonical 输出，可利用它完成精确相等检查，不必为每次中间计算创建永久 expression 对象。

hash、modular arithmetic 或数值过滤器只能在有严格证明的条件下作为加速步骤。hash 相同不是表达式相等的证明；浮点 residual 很小不是 residual 为零的证明。接受路径最终必须服从本规范的精确数学关系。

8.4 Fact 存储与内存统计

原草案给出的 8 B LiveFact 示意对应一种紧凑表达式身份与 scope 编码：

```
expr_handle:u32 | scope:u16 | kind:u8 | flags:u8
```

它不是文件 ABI，也不是所有 ExprView 的强制实现布局。static YIELD、kernel 生成的 assumption view 或独立 canonical model 可能需要另一种内部引用。live 索引、scope frame、allocator、scalar 解码缓存也有额外成本。

将历史 FactId 直接存入不可缩短的 vector，再把 dead 标记置位，仍然会保留与 total fact slot 数量成正比的表项。若希望 mutable fact 状态与 peak live facts 成正比，需要可回收的 live 索引或可回收分块表。FactId 保持逻辑 SSA；物理存储槽可以重复使用，但不可让旧 FactRef 指向新定义。

完整 mutable memory 至少包括

$$\text{mapping/obligations} + \text{live facts} + \text{scope journals} + \text{arithmetic workspace} + \text{bounded caches}.$$

只读模型和 RPX 映射也会占虚拟地址空间及实际 resident pages。统计必须分清逻辑 live fact 数、可变堆内存、mapped bytes 和 peak RSS；不能将其中一个当成另一个。

8.5 Batch 与高层 theorem 的边界

未来 batch opcode 必须有明确的有限 core 展开：

$$\mathrm{BATCH}(I_1, \ldots, I_n) \equiv I_1; \ldots; I_n.$$

展开不仅须保留数学命题，还须规定 operand 检查、FactId、scope、mapping、obligation 与失败行为。如果 fused encoding 省略临时物理 fact，它仍必须与规定的可观察逻辑状态一致。不能通过 batch 名称引入只能由额外 theorem 才能推出的结论。基础 v0.1 不支持这些保留 opcode；先使 core checker 正确，再另行发布 batch 扩展。

高层 theorem 可以缩短证明本身，其收益包括更少的 coefficient 访问而非仅更小的 dispatch 开销。它同时扩大可信逻辑边界，必须与编码优化分别统计、审阅和报告。

8.6 与 VIPR 的可比实验

VIPR 的证明本身也可以顺序验证，并携带最后引用信息以释放不再需要的 constraints。其证书整理工具可以构建依赖图与拓扑排序，不能把整理阶段的成本误算为每次基础验证的必要成本。[1]

RPX 的预期工程收益包括二进制解码、连续 sparse 存储、小整数快路径和结构化 scope 管理。这些是待验证的假设，不是现有 benchmark 结果。完整 reformulation verification 还承担双向 mapping 和输出 obligations；VIPR 对一个模型的目标界证明不能直接作为同命题的基准。

[1] K. K. H. Cheung, A. Gleixner, D. E. Steffy, *Verifying Integer Programming Results*, IPCO 2017. https://arxiv.org/abs/1611.08832

建议将同一份共同规则子集中的证明分别编码成 VIPR 与 RPX，保持推导、coefficients 和结论相同。对整数、小有理数和大有理数分别测量，并记录算术库、CPU、编译选项和线程数。单线程和多线程实验分别报告；VIPR 的当前项目提供并行 checker，不能以线程数不等的测试推导格式优势。[2]

测量项	必须明确的条件
端到端时间	模型读入、digest、loader、proof execution 是否全部计入。
验证时间	排除 assembler/证书整理时，单独报告被排除部分的成本。
I/O 与解码	cold/warm cache 分别报告；不能混用。
算术成本	coefficient 访问数、bit length 分布、promotion 和 gcd 工作量。
状态成本	split 数、深度、yield 数、mapping/obligation 变更与回滚。
内存	peak RSS、mapped bytes、live facts、累计事实数、工作区上限。
证书大小	完整文件，包括 pools、tapes、metadata；另报是否压缩。
重复性	固定证书集合、硬件、库版本和计时方法，报告多次重复的分布。

例如精确算术占总运行时间的 70% 时，只消除其他全部成本的加速上限为 $1/0.7 \approx 1.43$。这是 Amdahl 型的假设算例，不是 RPX 与 VIPR 的实测比值。只有共同证明实验通过后，再独立测量三个完整 reformulation 示例及更大模型，才能决定下一轮应优化解码、算术还是代换。

[2] https://github.com/scipopt/vipr

附录 A　修订提案、开放问题与冻结检查

A.1　提案登记

以下条目列出 D0 为使原草案可独立审阅而补充的候选规则。它们在相关章节中给出完整行为；本表不表示已通过审议。后续修改应保留编号，并记录接受、修改或撤回的决定。

编号	候选决定
P-01	目标统一为最小化；文件声明与调用者要求绑定；双向 EQ/LE 推出 optimum equivalence，F/G 不要求互逆。
P-02	每支从相同父状态开始；DONE 取可达支交集；不同 mapping 可封存为 SEALED_PIECEWISE；混合已定义/未定义立即拒绝。
P-03	FactId 按语法深度优先顺序分配，split 子支定义先于 merged outputs；不可达指令仍占 slot；DROP 仅释放当前 scope 的 live facts。
P-04	INT.AFF 允许任意有理数倍 EQ 项，同时保持整数 offset 和 INT multiplier 为整数。
P-05	CHECK 的 EXACT flag 由 kernel 验证恒真/结构整数性条件，不是 assembler 的成功声明；不分配可见 FactId。
P-06	验证 SPLIT.SIGN 两个覆盖范围；用于解重建时，零优先选择 le 支，以定义确定性映射。
P-07	明确 Header 偏移、ScalarPool、map 记录、全部 instruction payload、padding/count 和可选非逻辑 metadata 边界。

A.2　开放问题

编号	冻结前或独立工具发布前的决定
O-01	MCM 的独立 canonical 文件序列化、SHA 覆盖范围与 schema 版本。数学接口已定义，文件 ABI 尚未定义。
O-02	MPS 输入 dialect、decimal/二进制浮点来源语义、缺省 bounds、ranges、names、目标方向与 unsupported constructs 的完整转换规则。
O-03	面向数值解的独立 mapping 重建接口、输出格式与 metadata schema；本手册已有 guard/映射数学规则，但没有实现该工具。
O-04	Batch、theorem registry 与能力协商。基础 v0.1 一律拒绝保留 opcode；未来发布必须单独审阅。
O-05	资源限额的默认 profile、数值错误码和可重复 benchmark 数据集。格式硬边界已固定，部署限额尚未冻结。

O-01 与 O-02 阻止宣称完整文件工具链已经互通，但不阻止使用明确绑定的内存 canonical model 接口讨论和验证 RPX 语义。若最终发布采用该接口而不同时发布 MCM/MPS 工具，发布说明必须明确这一范围。

A.3 冻结检查点

1. 确认两个 profile、目标方向、offset 与调用者 policy，并证明 contract 定理的前提完整。
2. 接受或修订 branch 事务、三态 mapping、混合定义拒绝、obligation 交集及 root false 的规则。
3. 确认 FactId 编号、scope-local DROP、静态 YIELD 的 ExprView 与 exact proposition equality。
4. 确认全部 28 个 opcode 的字段、长度、flags、aux、类型前提与错误条件；不存在未定义的核心 mnemonic。
5. 确认 ScalarPool、ExprPool、Map、Header、Directory 的编码，特别是 count、padding、overflow 和 mandatory/optional 行为。
6. 逐条审阅第七章三个完整双向证明，使每个成功路径都有完整 mappings 和 obligations。
7. 为 wire format 准备 Header、ScalarRef、ScalarPool、每个 opcode、nested split 与 malformed 文件的 golden vectors。本手册给出的若干向量是起点，不能冒充覆盖全部 opcode 的测试套件。
8. 为接受结果、unsupported、resource limit 和 I/O failure 定义公开接口与诊断约定。
9. 确认核心规范不借助 proof search、浮点容差或隐藏 theorem 获得接受结果。
10. 所有接受的修订同步到示例与文档，再将草案状态改为 frozen v0.1；冻结后不改写已发布字段含义。

A.4 修订记录

日期	修订	内容
2026-10-09	D0	首次整理完整中文 LaTeX 手册；采用原始 28-opcode 核心，登记 P-01 至 P-07，提供完整双向示例和候选 wire format。

本文的专业排版与章节组织不构成规范已经成熟的证据。数学接受条件、字节级互通、失败案例和独立实现的一致性，才是下一阶段逐步审阅的对象。